

0장. 리눅스 서버 · 개발 환경 구축

이 장은 앞으로 다룰 13개의 취약점 실습 애플리케이션이 올라갈 토대를 만드는 장이다. 뒤에 나오는 모든 장은 "이 서버 위에 이런 앱을 하나 더 만든다"는 전제로 진행되므로, 여기서 만들어 둔 서버의 구조와 습관이 이후 모든 실습의 품질을 결정한다. 특히 이 교재는 컨테이너로 문제를 감싸지 않고 리눅스 서버에 각 구성요소를 직접 설치하는 방식을 택했다. 이유는 단순하다. 4장의 경로 조작, 6장의 운영체제 명령어 삽입, 7장의 파일 업로드처럼 "이 파일이 서버의 어느 경로에 어떤 권한으로 저장되는가", "이 프로세스는 어떤 계정으로 실행되고 있는가"가 취약점의 핵심을 이루는 실습이 많은데, 컨테이너 안에 모든 것을 숨겨버리면 이런 질문에 대한 감각이 무뎈다. 리눅스 서버에 직접 설치하면 파일 경로, 실행 계정, 권한, 네트워크 포트가 모두 눈앞에 드러나므로 실습의 현실감이 크게 올라간다.

0.1 과정 오리엔테이션

이 과정에서 무엇을 만드는가

이 교재의 목표물은 하나의 거대한 애플리케이션이 아니라, 서로 독립된 13개의 웹 애플리케이션이다. 각 애플리케이션은 하나의 취약점 유형(SQL 삽입, 코드 삽입, 경로 조작 등)을 다루며, 데모용 입력 폼 한 장으로 끝나는 게 아니라 로그인, 목록 조회, 검색, 상세 페이지, 관리자 화면처럼 실제 서비스가 갖추고 있을 법한 구성을 갖춘다. 예를 들어 2장에서 다룰 SQL 삽입 실습은 "임직원 검색 포털"이라는, 사내에서 실제로 쓰일 법한 형태의 사이트로 만들어진다. 이렇게 만드는 이유는 명확하다. 취약점은 진공 상태의 코드 한 줄에서 발생하는 게 아니라, 로그인 기능·검색 기능·게시판 기능처럼 실제 서비스의 구체적인 맥락 안에서 발생한다. 맥락 없이 취약한 코드 한 줄만 보여주면 "이런 코드는 안 쓰면 되지"라는 인상만 남고, 실제 개발 과정에서 왜 이런 실수가 반복되는지에 대한 감각을 얻지 못한다.

13개의 애플리케이션은 모두 같은 서버 위에서 서로 다른 포트로 동시에 실행된다. 2장의 임직원 검색 포털은 3001번 포트, 3장의 관리자 계산식 콘솔은 3002번 포트, 이런 식으로 3001번부터 3013번까지 순서대로 배정된다. 이렇게 한 서버에 여러 애플리케이션을 나란히 띄워 두면, 강의 중 필요할 때마다 브라우저 탭만 바꿔가며 여러 취약점을 비교하고 대조할 수 있다. 예를 들어 5장(XSS)과 12장(CSRF)을 다룰 때는 이 두 애플리케이션과는 별개로 공격자 역할을 하는 서버 하나가 4000번 포트에서 따로 돌아간다. 이 공격자 서버는 XSS로 탈취한 쿠키를 받아 기록하거나, CSRF 공격을 실행하는 함정 페이지를 호스팅하는 역할을 한다. 13장(SSRF)에서는 반대로 "외부에서는 절대 접근할 수 없는 내부망 서버" 역할을 하는 애플리케이션이 9090번 포트에서 방화벽 뒤에 숨어 실행된다.

서버 전체의 구조를 하나의 그림으로 표현하면 다음과 같다. 맨 앞단에는 Nginx가 80번 포트에서 모든 웹 요청을 받는다. Nginx는 요청을 들여다보고 어떤 애플리케이션으로 보낼지 판단한 뒤, 뒷단의 각 Node.js 애플리케이션(3001~3013번 포트)으로 요청을 전달한다. 각 애플리케이션은 데이터가 필요할 때 MySQL(3306번 포트)에 접속하고, 11장의 LDAP 실습 애플리케이션은 추가로 OpenLDAP 서버(389번 포트)에 접속한다. 13장의 애플리케이션만 예외적으로 내부망 역할을 하는 별도의 서버(9090번 포트)와 통신한다. 이 구조를 미리 그려두는 이유는, 이후 각 장에서 "왜 이 포트를 쓰는지", "이 요청이 어디서 어디로 가는지"를 매번 설명하지 않고 이 그림 하나로 대체하기 위함이다.

저장소 구조

실습 코드는 하나의 저장소 아래 장별로 디렉토리를 나누어 관리한다. 최상위 디렉터리 이름은 `secure-coding-labs`로 하고, 그 아래에 `lab01-sqli`, `lab02-code-injection`처럼 장 번호와 취약점 이름을 조합한 이름의 디렉토리를 둔다. 공격자 역할을 하는 서버는 별도로 `attacker-server`라는 이름의 디렉터리에 둔다. 이렇게 이름을 붙여두면 나중에 특정 장의 코드를 찾을 때 디렉터리 이름만 보고 바로 위치를 파악할 수 있고, 강의 중 여러 실습을 오갈 때도 혼동이 적다.

```

secure-coding-labs/
├── lab01-sqli/           # 2장 SQL 삽입 - 임직원 검색 포털
├── lab02-code-injection/ # 3장 코드 삽입 - 관리자 계산식 콘솔
├── lab03-path/          # 4장 경로 조작 및 자원 삽입
├── lab04-xss/           # 5장 XSS - 커뮤니티 게시판
├── lab05-os-command/    # 6장 운영체제 명령어 삽입
├── lab06-upload/        # 7장 파일 업로드
├── lab07-open-redirect/  # 8장 오픈 리다이렉트
├── lab08-xxe/           # 9장 XXE
├── lab09-xpath/         # 10장 XML 삽입
├── lab10-ldap/          # 11장 LDAP 삽입
├── lab11-csrf/          # 12장 CSRF
├── lab12-ssrf/          # 13장 SSRF
├── lab13-security-decision/ # 14장 보안기능 결정 부적절 입력값
├── attacker-server/     # 공격자 실습 서버 (attacker.local:4000)
└── internal-server/     # 내부망 역할 서버 (127.0.0.1:9090, 13장 전용)

```

각 **labXX** 디렉터리 내부의 구조는 0.7절에서 정하는 공통 스캐폴드를 그대로 따른다. 즉 어떤 장의 코드를 열어도 **routes**, **views**, **models** 같은 디렉터리가 같은 위치, 같은 역할로 존재한다. 이렇게 구조를 통일해 두면 3장에서 배운 디렉터리 구조를 4장에서 다시 설명할 필요가 없어지고, 수강생도 새 장을 시작할 때마다 익숙한 뼈대 위에 그날의 취약점만 얹어서 이해하면 된다.

실습 윤리와 법적 책임

이 교재에서 다루는 공격 기법은 모두 실제로 동작하는 코드이며, 이는 허가받지 않은 시스템에 사용될 경우 그대로 범죄가 된다. 정보통신망 이용촉진 및 정보보호 등에 관한 법률(정보통신망법) 제48조는 정당한 접근권한 없이 정보통신망에 침입하는 행위, 그리고 정보통신망의 안정적 운영을 방해할 목적으로 대량의 신호를 보내거나 부정한 명령을 처리하도록 하는 행위를 금지하고 있다. 여기서 다루는 SQL 삽입이나 명령어 삽입 같은 기법을 실습 목적이 아닌 실제 서비스에 사용하면 이 조항에 해당하는 범죄가 된다.

따라서 이 교재의 모든 공격 실습은 강사와 수강생이 직접 구축하고 통제하는 격리된 환경에서만 수행한다는 원칙을 처음부터 분명히 한다. 실습 서버는 인터넷에 공개된 프로덕션 환경이 아니라, 이 강의만을 위해 만든 가상 머신 혹은 사설 네트워크 안의 서버여야 한다. 수강생이 이후 실무에서 유사한 코드를 발견했을 때 여기서 배운 공격 기법을 무단으로 시도하는 일이 없도록, 이 원칙은 실습을 시작하기 전에 명확히 짚고 넘어가야 한다.

0.2 리눅스 서버 준비

서버의 형태 선택하기

실습 서버는 물리적으로 존재하는 컴퓨터일 수도 있고, VMware나 VirtualBox 위에 띄운 가상 머신일 수도 있으며, AWS나 네이버 클라우드 같은 클라우드 사업자가 제공하는 인스턴스일 수도 있다. 세 방식 모두 이 교재의 실습을 진행하는 데 문제가 없지만, 각각 장단점이 다르다.

물리 서버는 성능이 가장 안정적이고 네트워크 지연이 없다는 장점이 있지만, 실습 중 시스템을 심각하게 망가뜨렸을 때 되돌리기가 번거롭다. 가상 머신은 스냅샷 기능을 이용해 "환경 구축이 끝난 상태"를 저장해 두었다가, 실습 중 시스템이 꼬였을 때 그 시점으로 즉시 되돌릴 수 있다는 점에서 교육용으로 특히 유리하다. 클라우드 인스턴스는 초기 설정이 빠르고 여러 학생에게 동일한 환경을 나눠주기 쉽지만, 비용이 발생하고 네트워크 설정(보안 그룹 등)에 대한 추가 이해가 필요하다.

이 교재는 어떤 방식을 선택하든 적용 가능하도록 작성했으며, 강의 환경에 따라 적절히 선택하면 된다. 다만 실습에 사용할 서버는 최소 2개의 가상 CPU, 4GB의 메모리, 20GB의 디스크 공간을 확보하는 것을 권장한다. Node.js 애플리케이션 13개와 MySQL, Nginx, OpenLDAP까지 한 서버에 함께 올리기 때문에 이보다 낮은 사양에서는 여러 애플리케이션을 동시에 띄워 놓고 비교 시연을 할 때 응답이 느려질 수 있다.

배포판 선택

이 교재는 Ubuntu Server 22.04 LTS를 기준 배포판으로 삼는다. LTS는 Long Term Support의 약자로, 이 버전이 출시된 이후 장기간(우분투의 경우 5년)에 걸쳐 보안 패치가 제공됨을 의미한다. 실습용으로 최신 버전이 아닌 LTS 버전을 선택하는 이유는, 최신 기능보다 안정성과 문서화된 정보의 양이 우선이기 때문이다. LTS 버전은 오랫동안 널리 쓰여 왔기 때문에 온라인에 축적된 문제 해결 사례도 많다.

Ubuntu 계열이 아닌 RHEL(Red Hat Enterprise Linux) 계열 환경에서 실습해야 하는 경우를 위해, 이 교재는 대응하는 Rocky Linux 9의 명령어를 함께 표기한다. Ubuntu는 Debian 계열로 패키지 관리자로 **apt**를 사용하고, Rocky Linux는 Red Hat 계열로 **dnf**를 사용한다. 두 계열은 패키지 이름, 설정 파일의 위치, 기본 방화벽 도구(**ufw** vs **firewalld**)가 다르므로, 이 교재에서는 매 설치 단계마다 두 계열의 명령을 나란히 제시한다.

서버 기본 정보 확인

서버에 처음 접속하면 가장 먼저 이 서버가 어떤 상태인지 확인하는 습관을 들여야 한다. **hostnamectl** 명령은 호스트 이름, 운영체제 이름과 버전, 커널 버전, 아키텍처(x86_64 또는 ARM64) 등 시스템의 기본 정보를 한 번에 보여준다. 이 정보는 이후 특정 소프트웨어를 설치할 때 "이 서버 아키텍처에 맞는 패키지를 받아야 한다"는 판단의 근거가 된다.

```
hostnamectl
```

ip a 명령은 서버가 가진 네트워크 인터페이스 목록과 각 인터페이스에 할당된 IP 주소를 보여준다. 이 교재에서 만드는 애플리케이션들은 대부분 **127.0.0.1**(로컬호스트, 이 컴퓨터 자기 자신을 가리키는 특수 주소)에서 실행되지만, 외부에서 접속해 시연해야 하는 상황에서는 이 명령으로 확인한 IP 주소를 사용하게 된다.

```
ip a
```

df -h는 디스크 사용량을, **free -m**은 메모리 사용량을 사람이 읽기 쉬운 단위(-h는 human-readable, -m은 메가바이트 단위)로 보여준다. 실습 도중 디스크 공간이 부족해 MySQL이나 npm 설치가 실패하는 경우가 종종 있는데, 이런 문제를 사전에 방지하려면 작업을 시작하기 전에 여유 공간을 확인하는 습관이 필요하다.

```
df -h
free -m
```

시간대 설정

서버의 시간대가 맞지 않으면 이후 만들 애플리케이션의 로그 시각, 게시글 작성 시각 등이 실제 시간과 어긋나 혼란을 준다. 특히 이 교재의 실습에서는 로그를 보며 "언제 어떤 공격이 발생했는가"를 확인하는 과정이 반복되므로, 처음부터 시간대를 한국 표준시로 맞춰 두는 것이 좋다.

```
sudo timedatectl set-timezone Asia/Seoul
```

패키지 저장소 갱신과 시스템 업데이트

리눅스 배포판은 소프트웨어를 "저장소(repository)"라는 중앙 목록을 통해 설치한다. 이 저장소 목록은 시간이 지나면서 새로운 버전의 패키지 정보로 갱신되므로, 새 서버를 준비할 때는 가장 먼저 이 목록을 최신 상태로 맞추고 이미 설치된 패키지를 최신 버전으로 올려야 한다. 이는 단순한 습관이 아니라 보안과 직결되는 절차다. 오래된 패키지에는 이미 공개된 취약점이 남아 있을 수 있기 때문이다.

Ubuntu 계열에서는 다음 명령으로 저장소 목록을 갱신하고 설치된 패키지를 업그레이드한다.

```
sudo apt update
sudo apt upgrade -y
```

apt update는 저장소 서버에 접속해 "어떤 패키지의 어떤 버전이 있는지"에 대한 목록만 갱신하며, 실제로 무언가를 설치하거나 변경하지는 않는다. **apt upgrade**는 이 갱신된 목록을 기준으로 이미 설치된 패키지 중 새 버전이 있는 것을 실제로 업그레이드한다. **-y** 옵션은 설치 중 나오는 확인 질문에 자동으로 "예"라고 답하겠다는 뜻이다.

Rocky Linux 계열에서는 **dnf** 명령 하나로 저장소 정보 갱신과 업그레이드를 동시에 수행한다.

```
sudo dnf update -y
```

빌드 도구 설치

이후 이 교재에서 설치할 Node.js 패키지 중 일부(대표적으로 9장에서 사용할 **libxmljs**)는 자바스크립트로만 작성된 것이 아니라 내부에 C/C++로 작성된 코드를 포함하고 있으며, 설치할 때 이 부분을 서버에서 직접 컴파일해야 한다. 이 컴파일 작업을 수행하려면 컴파일러(**gcc**)와 빌드 도구(**make**)가 서버에 미리 설치되어 있어야 한다. 이런 도구가 없는 상태에서 해당 패키지를 설치하면 알 수 없는 에러 메시지와 함께 설치가 실패하는데, 원인을 모르면 상당히 당황스러운 상황이 된다. 따라서 이 시점에 미리 설치해 둔다.

```
# Ubuntu 계열
sudo apt install -y build-essential python3 git curl unzip

# Rocky 계열
sudo dnf groupinstall -y "Development Tools"
sudo dnf install -y python3 git curl unzip
```

build-essential은 Ubuntu에서 **gcc**, **g++**, **make** 등 컴파일에 필요한 도구를 한 번에 묶어 설치해주는 메타 패키지다. **python3**가 함께 설치되는 이유는, Node.js의 네이티브 모듈 빌드 도구인 **node-gyp**이 내부적으로 Python을 사용하기 때문이다. **git**은 소스 코드 버전 관리 도구이며, **curl**은 이후 Node.js 저장소를 등록하는 스크립트를 받아올 때 사용한다.

실습 전용 계정 생성

서버에 처음 접속하면 대개 **root** 계정이거나 클라우드 사업자가 만들어준 관리자 계정을 사용하게 된다. 하지만 이후 만들 모든 애플리케이션은 이 관리자 계정이 아닌, 이 실습을 위해 새로 만든 일반 계정으로 실행해야 한다. 이는 최소 권한 원칙(Principle of Least Privilege)이라는 보안의 기본 원칙을 실습 환경에서부터 체화하기 위함이다.

이 원칙이 왜 중요한지는 뒤에 나올 장들에서 실감하게 된다. 예를 들어 6장에서는 운영체제 명령어 삽입 취약점을 이용해 공격자가 서버에서 임의의 명령을 실행하는 상황을 재현하는데, 이때 애플리케이션이 **root** 권한으로 실행되고 있었다면 공격자는 서버 전체를 장악할 수 있다. 반면 애플리케이션이 권한이 제한된 일반 계정으로 실행되고 있었다면, 공격자가 명령을 실행하더라도 그 계정의 권한 범위 안에서만 피해가 발생한다. 즉 애플리케이션을 어떤 계정으로 실행하느냐는 취약점이 발생했을 때의 피해 범위를 결정하는 중요한 요소이며, 이 교재에서는 이를 처음부터 올바르게 설정해 둔다.

```
sudo adduser labuser
sudo usermod -aG sudo labuser # Rocky 계열은 sudo 대신 wheel 그룹: usermod -aG wheel labuser
```

adduser는 새 계정을 만들면서 홈 디렉터리 생성, 비밀번호 설정 등을 대화형으로 안내해주는 명령이다. **usermod -aG sudo labuser**는 이 계정을 **sudo** 그룹에 추가해, 필요할 때 **sudo** 명령을 통해 일시적으로 관리자 권한을 사용할 수 있게 해준다. 이후 서버 작업은 이 **labuser** 계정으로 접속해 진행하고, **root** 계정은 꼭 필요한 시스템 설정 변경 시에만 **sudo**를 통해 잠깐씩 사용한다.

SSH 접속 설정

원격 서버에 접속할 때 사용하는 SSH는 기본적으로 비밀번호 인증을 지원하지만, 비밀번호는 무차별 대입 공격(brute force)에 취약하다. 실무에서는 비밀번호 대신 키 쌍(공개키·개인키)을 이용한 인증을 사용하는 것이 표준이며, 이 교재에서도 이 방식을 따른다.

키 쌍은 자신의 컴퓨터(강사 또는 수강생의 로컬 PC)에서 생성한다.

```
ssh-keygen -t ed25519 -C "labuser@secure-coding"
```

이 명령을 실행하면 개인키(**id_ed25519**)와 공개키(**id_ed25519.pub**)가 한 쌍으로 생성된다. 개인키는 절대로 다른 사람과 공유하지 않고 자신의 컴퓨터에 보관하며, 공개키는 접속하려는 서버에 등록한다. 공개키를 서버에 등록하는 가장 간편한 방법은 **ssh-copy-id** 명령을 사용하는 것이다.

```
ssh-copy-id labuser@서버IP주소
```

윈도우에서 리눅스 서버에 키를 등록하기 위해 키를 직접 작성한다. 메모장 등으로 C:\Users\사용자\리눅스\ssh\id_rsa.pub 파일을 열어 안의 긴 텍스트를 복사한다. 리눅스 서버에 로그인한다. 리눅스에서 아래 명령어를 차례대로 입력해 키를 붙여넣는다.

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

```
echo "복사한공개키내용" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

키 기반 인증이 정상적으로 동작함을 확인한 뒤에는, 서버의 SSH 설정 파일(/etc/ssh/sshd_config)에서 비밀번호 인증과 root 계정의 직접 로그인을 차단한다. 이 두 가지 설정은 서버에 대한 무차별 대입 공격의 성공 가능성을 원천적으로 낮춘다.

```
PasswordAuthentication no
PermitRootLogin no
```

설정을 변경한 뒤에는 SSH 서비스를 재시작해야 적용된다.

```
sudo systemctl restart sshd
```

이 설정을 적용하기 전에 반드시 새 터미널 창에서 키 기반 접속이 정상적으로 되는지 먼저 확인해야 한다. 확인 없이 비밀번호 인증을 먼저 꺼버리면, 키 설정에 문제가 있을 경우 서버에 다시 접속할 방법이 없어지는 상황이 발생할 수 있다.

방화벽 기본 개념과 설정

방화벽은 서버로 들어오고 나가는 네트워크 트래픽을 규칙에 따라 허용하거나 차단하는 장치(또는 소프트웨어)다. 이 교재에서 만드는 서버는 웹 서비스(80·443번 포트), 실습용 애플리케이션(3001~3013번 포트), SSH(22번 포트)는 외부에서 접근할 수 있어야 하지만, MySQL(3306번 포트)이나 OpenLDAP(389번 포트) 같은 내부 인프라 포트는 외부에 노출되어서는 안 된다. 이런 포트가 외부에 열려 있으면, 데이터베이스 계정 정보가 유출되었을 때 외부 공격자가 직접 데이터베이스에 접속을 시도할 수 있는 통로가 생긴다.

Ubuntu 계열은 **ufw**(Uncomplicated Firewall)라는 비교적 단순한 방화벽 도구를 기본으로 제공한다.

```
sudo ufw allow 22/tcp          # SSH
sudo ufw allow 80/tcp          # HTTP (Nginx)
sudo ufw allow 443/tcp         # HTTPS
sudo ufw allow 3001:3013/tcp    # 실습 애플리케이션 (직접 접근 시연용)
sudo ufw enable
sudo ufw status
```

ufw allow는 지정한 포트에 들어오는 트래픽을 허용하는 규칙을 추가한다. **3001:3013/tcp**처럼 콜론으로 범위를 지정하면 그 사이의 모든 포트가 한 번에 허용된다. **ufw enable**은 지금까지 추가한 규칙을 실제로 활성화하는 명령이며, 이 명령을 실행하기 전에 반드시 SSH 포트(22번)를 허용 목록에 넣어 두어야 한다. 그렇지 않으면 방화벽이 켜지는 순간 서버에 대한 SSH 접속 자체가 끊어질 수 있다. **ufw status**로 현재 적용된 규칙을 확인할 수 있다.

Rocky Linux 계열은 **firewalld**를 사용한다.

```
sudo firewall-cmd --permanent --add-port=22/tcp
sudo firewall-cmd --permanent --add-port=80/tcp
sudo firewall-cmd --permanent --add-port=443/tcp
```

```
sudo firewall-cmd --permanent --add-port=3001-3013/tcp
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

--permanent 옵션은 이 규칙을 재부팅 후에도 유지되도록 영구 설정에 반영하겠다는 뜻이며, **--reload**를 실행해야 이 영구 설정이 실제로 적용된다.

MySQL(3306번)과 OpenLDAP(389번) 포트는 이 방화벽 규칙에 포함시키지 않는다. 즉 명시적으로 허용하지 않음으로써 기본값(거부)이 유지되도록 두며, 이 두 서비스는 오직 같은 서버 안에서(127.0.0.1을 통해) Node.js 애플리케이션이 접근하도록 뒤에서 설정한다.

SELinux와 AppArmor 개념

방화벽이 네트워크 계층에서 트래픽을 통제한다면, SELinux(Security-Enhanced Linux)와 AppArmor는 한 단계 더 들어가 "어떤 프로세스가 어떤 파일과 리소스에 접근할 수 있는가"를 통제하는 강제 접근 통제(Mandatory Access Control) 시스템이다. Rocky Linux를 포함한 RHEL 계열은 기본적으로 SELinux가 활성화되어 있고, Ubuntu는 AppArmor를 기본으로 사용한다.

이 두 시스템은 애플리케이션이 예상치 못한 파일이나 포트에 접근하려 할 때 이를 차단해, 설령 애플리케이션 코드에 취약점이 있어 공격자가 임의 명령을 실행하는 데 성공하더라도 그 피해가 시스템 전체로 번지는 것을 막아주는 추가 방어선 역할을 한다. 예를 들어 SELinux가 활성화된 상태에서는 Nginx 프로세스가 기본적으로 허용된 디렉터리 밖의 파일에 접근하려 하면 거부당한다. 이 특성은 실습 중 "분명히 파일 권한은 맞는데 접근이 안 된다"는 상황의 원인이 될 수 있으므로, 이 시점에 개념만 짚어두고 실제로 이 문제를 만났을 때는 0.9절의 트러블슈팅을 참고한다.

이 교재의 실습에서는 SELinux나 AppArmor를 완전히 꺼서 문제를 회피하는 방식을 권장하지 않는다. 실제 운영 환경에서는 이런 보호 장치를 끄는 것 자체가 보안 사고의 원인이 되기 때문이다. 대신 문제가 발생했을 때 로그(**ausearch**, **journalctl**)를 통해 어떤 접근이 차단되었는지 확인하고, 필요한 만큼만 예외 규칙을 추가하는 방식으로 접근한다.

실습 디렉터리 표준 구조 설계

애플리케이션 코드, 업로드된 파일, 로그 파일은 각각 성격이 다르므로 서로 다른 디렉터리에 분리해 저장한다. 이렇게 분리하는 이유는 단순히 정리를 위한 것이 아니라, 7장(파일 업로드)에서 다시 강조하듯 "업로드된 파일이 저장되는 위치"와 "애플리케이션 코드가 실행되는 위치"가 분리되어 있어야 업로드된 파일이 실행 가능한 스크립트로 둔갑하는 사고를 막을 수 있기 때문이다.

```
sudo mkdir -p /opt/labs
sudo mkdir -p /var/lab-uploads
sudo mkdir -p /var/log/labs
sudo chown -R labuser:labuser /opt/labs /var/lab-uploads /var/log/labs
```

/opt/labs는 애플리케이션 코드가 위치하는 디렉터리다. 리눅스에서 **/opt**는 관례적으로 시스템 기본 패키지가 아닌, 별도로 설치된 애플리케이션 소프트웨어를 두는 자리다. **/var/lab-uploads**는 7장의 파일 업로드 실습을 포함해 사용자가 서버에 업로드하는 파일이 저장되는 위치이며, **/var/log/labs**는 각 애플리케이션의 실행 로그가 기록되는 위치다. **chown -R labuser:labuser**는 이 세 디렉터리와 그 안의 모든 내용물의 소유자를 앞서 만든 실습 계정으로 지정해, 이 계정이 해당 디렉터리에 자유롭게 읽고 쓸 수 있도록 한다.

0.3 Node.js 설치

Node.js란

Node.js는 구글이 개발한 자바스크립트 엔진인 V8을 브라우저 밖에서도 동작하도록 감싼 실행 환경이다. 원래 자바스크립트는 웹 브라우저 안에서 HTML 페이지를 동적으로 조작하기 위해 만들어진 언어였고, 브라우저를 벗어나서는 실행될 방법이 없었다. Node.js는 이 자바스크립트 언어를 브라우저라는 울타리 밖으로 꺼내어, 서버 컴퓨터에서 파일을 읽고 쓰거나, 데이터베이스에 접속하거나, 네트워크 요청을 처리하는 등 일반적인 서버 프로그램이 하는 일을 자바스크립트로 할 수 있게 만들었다.

Node.js를 알아야 하는 이유

이 교재가 Node.js를 실습 환경으로 선택한 이유는 두 가지다. 첫째, 원본 가이드인 「JavaScript 시큐어코딩 가이드」의 모든 서버측 코드 예제가 Node.js와 ExpressJS(Node.js 위에서 동작하는 웹 프레임워크)를 기준으로 작성되어 있기 때문에, 원문의 코드를 그대로 이해하고 확장하려면 Node.js에 대한 이해가 필수적이다. 둘째, 자바스크립트라는 하나의 언어로 브라우저에서 실행되는 클라이언트 코드(VanillaJS, ReactJS)와 서버에서 실행되는 코드(ExpressJS)를 모두 작성할 수 있어, 5장(XSS)이나 12장(CSRF)처럼 클라이언트와 서버 양쪽의 코드가 함께 취약점을 만드는 상황을 하나의 언어로 일관되게 설명할 수 있다.

런타임 개념: V8 엔진과 이벤트 루프

Node.js의 핵심에는 V8이라는 자바스크립트 엔진이 있다. V8은 자바스크립트로 작성된 코드를 기계어로 변환해 실행하는 프로그램이며, 원래 크롬 브라우저 안에서 웹페이지의 자바스크립트를 실행하기 위해 만들어졌다. Node.js는 이 V8 엔진에 파일 시스템 접근, 네트워크 통신 같은 서버 환경에 필요한 기능을 추가로 붙여서 만들어졌다.

Node.js를 이해하는 데 있어 가장 중요한 개념은 이벤트 루프(Event Loop)다. 대부분의 전통적인 서버 프로그램은 요청 하나마다 스레드를 하나씩 배정해 처리하는데, Node.js는 기본적으로 하나의 메인 스레드에서 동작하며 이 이벤트 루프라는 메커니즘을 통해 여러 요청을 동시에 처리하는 것처럼 보이게 만든다. 예를 들어 데이터베이스에 쿼리를 보내고 응답을 기다리는 동안, Node.js는 이 대기 시간에 다른 요청을 처리한다. 응답이 도착하면 이벤트 루프가 이를 감지해 원래 대기하던 코드를 이어서 실행한다. 이 구조 덕분에 Node.js는 적은 자원으로 많은 수의 동시 연결을 처리하는 데 유리하다.

이 이벤트 루프 구조는 보안적으로도 중요한 의미를 갖는다. 3장에서 다룰 코드 삽입 취약점의 증상 중 하나로 "무한 루프로 인한 서비스 정지"를 다루게 되는데, 이는 Node.js가 단일 스레드로 동작하기 때문에 발생하는 현상이다. 하나의 요청 처리 코드가 무한히 반복되면 이벤트 루프 전체가 멈추고, 그 서버에 접속한 다른 모든 사용자의 요청도 함께 멈춰 버린다. 이런 특성을 미리 이해해 두면 이후 실습에서 마주치는 증상을 훨씬 자연스럽게 받아들일 수 있다.

LTS 버전과 최신 버전의 차이

Node.js 프로젝트는 짝수 번호의 메이저 버전(18, 20, 22 등)을 LTS(Long Term Support) 버전으로, 홀수 번호(19, 21 등)를 최신 기능이 먼저 반영되는 실험적 버전으로 관리한다. LTS 버전은 출시 후 약 30개월에 걸쳐 보안 패치와 버그 수정을 지속적으로 제공하며, 프로덕션 환경에서 사용하도록 권장되는 버전이다. 이 교재의 실습 서버에는 짝수 번호의 최신 LTS 버전을 설치한다. 특정 버전 번호는 npm 패키지의 호환성에 영향을 줄 수 있으므로, 실습을 시작하기 전 Node.js 공식 사이트에서 현재 LTS로 지정된 버전을 확인하는 것이 좋다.

설치 방법

Node.js를 설치하는 방법에는 여러 가지가 있으며, 각각 적합한 상황이 다르다.

NodeSource 저장소를 이용한 설치는 실습 서버처럼 특정 버전을 고정해 안정적으로 운영해야 하는 환경에 적합하다. NodeSource는 Node.js 공식 배포 파트너로, 배포판의 패키지 관리자([apt](#), [dnf](#))를 통해 최신 버전의 Node.js를 설치할 수 있도록 저장소를 제공한다.

```
# Ubuntu 계열 (Node.js 20.x LTS 기준)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Rocky 계열
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf install -y nodejs
```

첫 번째 줄은 NodeSource가 제공하는 설정 스크립트를 다운로드해 실행하는 것으로, 이 스크립트가 시스템의 패키지 저장소 목록에 NodeSource 저장소를 추가해준다. `-E` 옵션은 `sudo`로 실행하더라도 현재 사용자의 환경변수를 유지하라는 뜻이다. 이 작업이 끝나면 두 번째 줄에서 평소 사용하던 패키지 관리자 명령으로 Node.js를 설치할 수 있다.

nvm(Node Version Manager)을 이용한 설치는 여러 버전의 Node.js를 사용자 계정 안에서 자유롭게 전환하며 사용해야 할 때 유리하다. 예를 들어 어떤 실습에서는 특정 버전의 Node.js가 필요하고 다른 실습에서는 다른 버전이 필요한 상황이라면, nvm을 통해 버전을 즉시 전환할 수 있다.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.bashrc
nvm install --lts
nvm use --lts
```

배포판 기본 저장소를 이용한 설치(`apt install nodejs`를 NodeSource 없이 바로 실행하는 방식)는 가장 간단해 보이지만 이 교재에서는 권장하지 않는다. Ubuntu나 Rocky의 기본 저장소에 포함된 Node.js 버전은 대체로 최신 LTS 보다 오래되었으며, 배포판의 보안 업데이트 주기에 맞춰서만 갱신되기 때문에 최신 npm 패키지와 호환되지 않는 경우가 잦다.

세 가지 방식을 표로 정리하면 다음과 같다.

설치 방식	장점	단점	적합한 상황
NodeSource 저장소	최신 LTS 설치 가능, 시스템 전역에 적용, <code>systemctl</code> 과 잘 연동됨	시스템 전역 설정이라 버전 전환이 번거로움	이 교재의 실습 서버(고정된 버전으로 상시 서비스 운영)
nvm	사용자별로 여러 버전 자유롭게 전환	시스템 서비스(systemd)에서 사용하려면 경로를 명시해야 함	개인 개발 환경, 여러 프로젝트를 오가는 개발자
배포판 기본 저장소	명령 한 줄로 설치, 의존성 관리가 배포판에 통합	버전이 오래됨	버전에 민감하지 않은 간단한 스크립트 실행

이 교재는 서버 전체에서 하나의 고정된 버전을 상시 서비스로 운영하는 형태이므로 NodeSource 저장소 방식을 기본으로 채택하고, 개인 실습 환경에서 버전을 자주 바꿔야 하는 경우에 한해 nvm을 보조적으로 사용한다.

설치 확인

설치가 끝나면 버전 정보를 확인해 정상적으로 설치되었는지 검증한다.

```
node -v      # 예: v20.17.0
npm -v       # 예: 10.8.2
which node   # Node.js 실행 파일의 경로 확인
```

`node -v`와 `npm -v`는 각각 Node.js 런타임과 npm 패키지 관리자의 버전을 출력한다. `which node`는 현재 셸에서 `node` 명령이 실제로 어느 경로의 실행 파일을 가리키고 있는지 보여주는데, 이는 여러 설치 방식을 시도하다가 의도치 않게 서로 다른 버전이 충돌하는 문제를 진단할 때 유용하다.

자주 발생하는 설치 오류

PATH 미반영: nvm으로 설치한 직후 새 터미널을 열지 않고 계속 같은 세션에서 `node` 명령을 실행하면 "명령을 찾을 수 없다"는 오류가 발생할 수 있다. 이는 nvm이 셸 설정 파일(`.bashrc` 등)에 추가한 PATH 설정이 현재 세션에는 아직 반영되지 않았기 때문이며, `source ~/.bashrc` 명령으로 즉시 반영하거나 터미널을 새로 열면 해결된다.

프록시 환경: 사내망처럼 인터넷 접속에 프록시를 거쳐야 하는 환경에서는 `npm install`이 원격 저장소에 접속하지 못해 멈추는 경우가 있다. 이때는 npm에 프록시 서버 주소를 알려주어야 한다.

```
npm config set proxy http://프록시주소:포트
npm config set https-proxy http://프록시주소:포트
```

전역 설치 권한 문제(EACCES): `sudo` 없이 `npm install -g`로 전역 패키지를 설치하려 할 때 권한 부족 오류(EACCES)가 발생하는 경우가 있다. NodeSource 방식으로 설치한 Node.js는 시스템 전역 디렉터리에 설치되어 일반 사용자 권한으로 그 디렉터리에 쓰기 접근을 할 수 없기 때문이다. 이 문제는 `sudo npm install -g`로 우회할 수도 있지만, 더 나은 해결책은 npm의 전역 설치 경로를 사용자 홈 디렉터리 아래로 바꾸는 것이다.

0.4 npm과 프로젝트 구조

npm이란, 패키지 매니저의 역할

npm(Node Package Manager)은 Node.js와 함께 설치되는 패키지 관리 도구로, 다른 개발자가 만들어 공개한 코드 조각(패키지)을 손쉽게 내려받아 프로젝트에서 사용할 수 있게 해준다. 이 교재에서 다룰 `express`, `mysql`, `sequelize`, `socket.io` 같은 라이브러리는 모두 npm을 통해 설치한다. npm이 관리하는 패키지 저장소(npm 레지스트리)에는 전 세계 개발자들이 만든 수백만 개의 패키지가 등록되어 있으며, 이는 개발 속도를 크게 높여주지만 동시에 15장에서 다시 논의할 공급망 보안 문제의 근원이기도 하다. 내가 직접 작성하지 않은 코드를 신뢰하고 프로젝트에 끌어다 쓰는 것이기 때문이다.

프로젝트 초기화

lab01-sqli 디렉토리를 생성하여 진행한다. 새로운 Node.js 프로젝트를 시작할 때는 해당 디렉터리에서 `npm init` 명령을 실행한다.

```
mkdir lab01-sqli
cd lab01-sqli
```

```
npm init -y
```

`-y` 옵션은 프로젝트 이름, 버전, 설명 등을 묻는 질문들에 모두 기본값으로 답하고 즉시 `package.json` 파일을 생성하라는 뜻이다. 이 파일이 바로 이 디렉터리가 하나의 Node.js 프로젝트를 나타내는 표식이자 설정 파일이다.

package.json 구성 요소

`package.json`은 다음과 같은 형태를 갖는다.

```
{
  "name": "lab01-sqli",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mysql": "^2.18.1",
    "sequelize": "^6.37.3"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

`name`은 이 프로젝트의 이름이며, `main`은 이 프로젝트를 모듈로 불러왔을 때 진입점이 되는 파일을 가리킨다. `scripts`는 `npm run` 이름 형태로 실행할 수 있는 명령어 단축키를 정의하는 영역이다. `dependencies`와 `devDependencies`는 이 프로젝트가 의존하는 외부 패키지 목록이다.

의존성 개념: dependencies vs devDependencies

`dependencies`에는 애플리케이션이 실제로 동작하는 데 필요한 패키지(예: `express`, `mysql`)를 등록한다. 이 패키지들은 애플리케이션을 배포하거나 운영 서버에 올릴 때도 반드시 함께 설치되어야 한다. 반면 `devDependencies`에는 개발 과정에서만 필요하고 실제 운영 환경에서는 필요 없는 도구(예: 코드 변경을 감지해 자동으로 서버를 재시작해주는 `nodemon`)를 등록한다. 이 구분은 운영 서버에 불필요한 패키지가 설치되는 것을 막아 공격 표면(공격자가 악용할 수 있는 지점의 총량)을 줄이는 데 기여한다.

```
npm install express          # dependencies에 추가
npm install --save-dev nodemon # devDependencies에 추가
```

node_modules와 package-lock.json

`npm install`을 실행하면 `package.json`에 명시된 패키지들이 실제로 다운로드되어 `node_modules`라는 디렉터리에 저장된다. 이 디렉터리에는 직접 설치를 요청한 패키지뿐 아니라, 그 패키지들이 다시 의존하는 또 다른 패키지들까지 모두 함께 내려받아지므로 순식간에 수백, 수천 개의 하위 디렉터리로 불어난다. 이 때문에 `node_modules`는 일반적으로 버전 관리 시스템(Git)에 포함시키지 않는다.

`package-lock.json`은 `npm install`을 실행할 때 자동으로 생성되는 파일로, 이 프로젝트가 설치한 모든 패키지(직접 의존성과 간접 의존성 모두)의 정확한 버전을 기록해 둔다. 이 파일이 있으면 다른 컴퓨터나 다른 시점에 같은 프로젝트를 `npm install`했을 때도 정확히 같은 버전의 패키지 조합이 재현된다. 이는 "내 컴퓨터에서는 됐는데 서버에서는 안 된다"는 상황을 방지하는 데 중요한 역할을 하므로, `package-lock.json`은 `node_modules`와 달리 반드시 버전 관리에 포함시켜야 한다.

시맨틱 버저닝

`package.json`의 `dependencies`에 적힌 `^4.19.2` 같은 표기를 시맨틱 버저닝(Semantic Versioning)이라 부른다. 버전 번호는 `주버전.부버전.패치버전` 형식을 따르며, 주버전은 기존 사용법과 호환되지 않는 큰 변경이 있을 때, 부버전은 기존 사용법과 호환되면서 기능이 추가될 때, 패치버전은 버그 수정이 있을 때 올라간다. 버전 앞의 `^` 기호는 "주버전은 고정하되 그보다 낮은 자리는 최신 버전을 허용한다"는 뜻이며(예: `^4.19.2`는 `4.x.x` 범위 내 최신을 허용), `~` 기호는 "부버전까지 고정하고 패치버전만 최신을 허용한다"는 더 보수적인 범위를 뜻한다.

패키지 설치·삭제·업데이트

```
npm install 패키지명      # 설치 (dependencies에 추가)
npm uninstall 패키지명    # 삭제
npm update 패키지명       # package.json의 버전 범위 내에서 최신으로 업데이트
npm list                  # 현재 설치된 패키지 목록 확인
npm outdated              # 업데이트 가능한 패키지 확인
```

공급망 보안: npm audit

`node_modules`에 들어가는 코드는 대부분 내가 직접 작성하지 않은, 낯선 개발자가 작성한 코드다. 이 코드 중 일부에서 이미 알려진 보안 취약점이 발견되는 경우가 실제로 자주 있다. npm은 이런 위험을 점검할 수 있는 `npm audit` 명령을 기본으로 제공한다.

```
npm audit
```

이 명령을 실행하면 현재 프로젝트가 의존하는 패키지들 가운데 공개된 취약점 데이터베이스에 등록된 항목이 있는지 검사하고, 있다면 그 심각도(낮음/중간/높음/치명적)와 함께 보고해준다.

```
npm audit fix
```

이 명령은 발견된 취약점 중 상위 버전으로 올리는 것만으로 해결 가능한 것들을 자동으로 업데이트해준다. 다만 주버전이 바뀌는 큰 업데이트가 필요한 경우에는 자동으로 처리하지 않고 별도의 확인을 요구하는데, 이는 주버전 업데이트가 기존 코드의 동작을 깨뜨릴 수 있기 때문이다.

이 교재의 뒷부분에서 다시 다루겠지만, 원본 가이드가 작성된 2023년 이후로 시간이 흐르면서 가이드에 등장하는 일부 패키지(`csurf`, `request` 등)는 유지보수가 중단되었다. `npm audit`을 실행 초반부터 습관화해 두면, 이런 낡은 패키지에 대한 경고를 스스로 인지하고 대응하는 감각을 기를 수 있다.

실행 스크립트와 nodemon

앞서 `package.json`의 `scripts` 항목에서 본 것처럼, `npm start`는 `node app.js`를 실행하는 단축키로 정의할 수 있다. 개발 중에는 코드를 수정할 때마다 서버를 수동으로 켜다 켜는 대신, 파일 변경을 감지해 자동으로 재시작해주는 `nodemon`을 사용하면 편리하다.

```
npm install --save-dev nodemon
npx nodemon app.js
```

다만 `nodemon`은 어디까지나 개발 편의를 위한 도구이며, 실습 서버에 상시로 애플리케이션을 띄워 둘 때는 0.8절에서 다룬 `systemd`를 통해 관리한다.

0.5 MySQL 설치 및 설정

MySQL이란, 이 과정에서 사용하는 이유

MySQL은 세계에서 가장 널리 쓰이는 오픈소스 관계형 데이터베이스 관리 시스템(RDBMS) 중 하나다. 이 교재의 2장(SQL 삽입)을 포함해 대부분의 실습 애플리케이션이 사용자 정보, 게시글, 임직원 명단 같은 데이터를 저장하고 조회하는 데 이 데이터베이스를 사용한다. 특히 2장의 실습은 애초에 "데이터베이스에 보내는 쿼리문을 조작하는 공격"을 다루므로, 실제로 동작하는 데이터베이스가 있어야만 취약점의 증상(다른 사용자의 데이터가 보이거나, 인증이 우회되는 등)을 눈으로 확인할 수 있다.

설치

```
# Ubuntu 계열 (MySQL 8.0 기준)
sudo apt install -y mysql-server

# Rocky 계열
sudo dnf install -y mysql8.4-server
```

서비스 관리

설치가 끝나면 MySQL은 `systemd`가 관리하는 서비스로 등록된다. 서비스를 시작하고, 부팅 시 자동으로 실행되도록 설정한다.

```
sudo systemctl enable --now mysql # Rocky는 서비스명이 mysqld인 경우가 있음:
mysqld
sudo systemctl status mysql
```

`enable --now`는 "지금 즉시 서비스를 시작하고, 앞으로 서버가 재부팅될 때도 자동으로 시작하도록 등록한다"는 두 가지 동작을 한 번에 수행하는 옵션 조합이다. `status`는 현재 이 서비스가 정상적으로 실행 중인지, 혹시 오류로 멈춰

있는지를 보여준다. MySQL 관련 로그는 `/var/log/mysql/error.log`에 기록되므로, 서비스가 예상대로 시작되지 않을 때는 이 로그 파일을 먼저 확인한다.

초기 보안 설정

MySQL을 갓 설치한 상태는 익명 사용자 계정이나 테스트용 데이터베이스가 기본으로 포함되어 있는 등 보안상 다듬어지지 않은 상태다. MySQL은 이런 기본 설정을 정리해주는 대화형 스크립트를 제공한다.

```
sudo mysql_secure_installation
```

이 스크립트를 실행하면 순서대로 몇 가지 질문을 받는다. 먼저 비밀번호 검증 정책(VVALIDATE PASSWORD)을 사용할지 묻는데, 실습 환경에서는 너무 엄격한 정책이 오히려 불편할 수 있으므로 상황에 맞게 선택한다. 이어서 `root` 계정의 비밀번호를 설정하라는 질문이 나오는데, 이 비밀번호는 반드시 기억해 두어야 이후 데이터베이스 관리 작업을 진행할 수 있다. 그다음에는 익명 사용자 계정을 제거할지, `root` 계정으로 원격(다른 컴퓨터)에서 로그인하는 것을 금지할지, 기본으로 제공되는 `test` 데이터베이스를 제거할지, 지금까지의 변경사항을 즉시 적용하기 위해 권한 테이블을 다시 불러올지를 차례로 묻는다. 이 질문들에는 모두 "예"로 답하는 것이 안전한 기본 구성이다. 특히 원격 `root` 로그인을 금지하는 것은, 설정 나중에 방화벽 설정에 실수가 있어 3306번 포트가 외부에 노출되더라도 가장 강력한 권한을 가진 계정으로의 직접 침투를 막아주는 마지막 안전장치가 된다.

```
sudo mysql_secure_installation
```

```
Securing the MySQL server deployment.
```

```
Connecting to MySQL using a blank password.
```

```
The 'validate_password' component is installed on the server.
```

```
The subsequent steps will run with the existing configuration  
of the component.
```

```
Please set the password for root here.
```

```
New password:
```

```
Re-enter new password:
```

```
Estimated strength of the password: 100
```

```
Do you wish to continue with the password provided?(Press y|Y for Yes, any other  
key for No) : y
```

```
By default, a MySQL installation has an anonymous user,  
allowing anyone to log into MySQL without having to have  
a user account created for them. This is intended only for  
testing, and to make the installation go a bit smoother.  
You should remove them before moving into a production  
environment.
```

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y  
Success.
```

```
Normally, root should only be allowed to connect from  
'localhost'. This ensures that someone cannot guess at
```

the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
Success.

By default, MySQL comes with a database named 'test' that anyone can access. This is also intended only for testing, and should be removed before moving into a production environment.

Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y

- Dropping test database...

Success.

- Removing privileges on test database...

Success.

Reloading the privilege tables will ensure that all changes made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!

설정 파일 구조와 주요 옵션

MySQL의 설정은 Ubuntu 기준 `/etc/mysql/mysql.conf.d/mysqld.cnf` 파일, RockyLinux 기준 `/etc/my.cnf.d/mysql-server.cnf`에서 관리된다. 이 교재의 실습에서 주목해야 할 설정 항목은 다음과 같다.

```
[mysqld]
bind-address = 127.0.0.1
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci
port = 3306
```

`bind-address`는 MySQL이 어떤 네트워크 주소로 들어오는 연결을 받을지 지정하는 항목이다. 이를 `127.0.0.1`(로컬 호스트)로 설정하면, MySQL은 오직 같은 서버 안에서 오는 연결만 받아들이고 외부 네트워크에서 들어오는 연결은 아예 받지 않는다. 이는 0.2절에서 설정한 방화벽 규칙과 함께, 데이터베이스를 외부로부터 이중으로 격리하는 조치다. `character-set-server`를 `utf8mb4`로 지정하는 이유는, 기본 `utf8` 인코딩이 일부 이모지나 특수 문자를 제대로 저장하지 못하는 한계가 있기 때문에, 이를 완전히 지원하는 `utf8mb4`를 명시적으로 사용하는 것이 요즘의 표준이기 때문이다.

설정을 변경한 뒤에는 서비스를 재시작해야 반영된다.

```
sudo systemctl restart mysqld
```

데이터베이스와 계정 생성

MySQL에 접속해 이 교재 전체에서 공용으로 사용할 데이터베이스와, 애플리케이션이 사용할 전용 계정을 만든다.

```
sudo mysql -u root -p
```

MySQL 프롬프트에 접속한 뒤 다음 SQL 문을 순서대로 실행한다.

```
CREATE DATABASE lab CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
CREATE USER 'lab'@'localhost' IDENTIFIED BY '원하는비밀번호';
```

권한(GRANT) 개념과 최소 권한 원칙

계정을 만들었다면, 이 계정에게 어떤 작업을 허용할지 명시적으로 부여해야 한다. MySQL은 기본적으로 새로 만든 계정에 아무 권한도 주지 않으므로, **GRANT** 문으로 필요한 권한만 골라 부여한다.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON lab.* TO 'lab'@'localhost';  
FLUSH PRIVILEGES;
```

이 예제에서 **lab** 계정에게는 데이터를 조회(SELECT)하고, 추가(INSERT)하고, 수정(UPDATE)하고, 삭제(DELETE)하는 권한만 부여했다. 테이블 구조 자체를 바꾸는 **ALTER**, 테이블을 통째로 삭제하는 **DROP**, 서버의 파일 시스템에 접근하는 **FILE** 같은 권한은 의도적으로 제외했다. 이렇게 애플리케이션이 실제 업무에 필요한 최소한의 권한만 갖도록 하는 것이 최소 권한 원칙이며, 이는 2.4절(SQL 삽입 시큐어 코딩)에서 다시 그 의미를 되짚게 된다. 만약 애플리케이션 계정에 **DROP** 권한까지 부여되어 있었다면, SQL 삽입 취약점을 통해 공격자가 테이블 전체를 삭제하는 최악의 시나리오까지 가능해진다. 반대로 필요한 권한만 부여해 두면, 실령 SQL 삽입에 성공하더라도 공격자가 할 수 있는 일의 범위가 크게 줄어든다.

데이터베이스 권한 부여

```
GRANT ALL PRIVILEGES ON lab.* TO 'lab'@'localhost';  
FLUSH PRIVILEGES;
```

인증 플러그인과 Node 드라이버 연결 이슈

MySQL 8.0은 기본 인증 방식으로 **caching_sha2_password**를 사용하는데, Node.js의 구형 **mysql** 드라이버 패키지 중 일부 버전은 이 인증 방식을 지원하지 않아 연결 오류가 발생하는 경우가 있다. 이 문제를 만났을 때는 계정의 인증 방식을 예전부터 널리 호환되던 **mysql_native_password**로 바꿔주는 방법으로 해결할 수 있다. 다만 MySQL 8.4에서는 **mysql_native_password**가 기본적으로 비활성화되어 있으므로, 단순히 **ALTER USER ... IDENTIFIED WITH**

mysql_native_password만 실행하면 안 될 수 있다. 먼저 MySQL 설정에서 mysql_native_password를 활성화해야 한다.

```
[mysqld]
mysql_native_password=ON
```

```
sudo systemctl restart mysqld
```

```
sudo mysql -u root -p
```

```
ALTER USER 'lab'@'localhost' IDENTIFIED WITH mysql_native_password BY '원하는비밀번호';
FLUSH PRIVILEGES;
```

이는 어디까지나 구형 드라이버와의 호환을 위한 임시 조치이며, 사용하는 Node.js MySQL 드라이버가 `caching_sha2_password`를 지원하는 최신 버전이라면 이 조치가 필요하지 않을 수 있다. 연결 오류가 발생했을 때 진단 순서로 참고하면 된다.

스키마·시드 데이터 적재

각 실습 애플리케이션은 저마다 필요한 테이블 구조(스키마)와 초기 데이터(시드 데이터)를 SQL 파일로 관리한다. 이 파일들은 다음과 같이 명령줄에서 한 번에 적재할 수 있다.

```
mysql -u lab -p lab < db/schema.sql
mysql -u lab -p lab < db/seed.sql
```

연결 테스트

데이터베이스 설정이 끝나면 CLI에서 접속이 되는지 먼저 확인한다.

```
mysql -u lab -p -h 127.0.0.1 lab
```

CLI 접속이 확인되면, 실제로 Node.js 애플리케이션에서도 정상적으로 연결되는지 짧은 테스트 스크립트로 검증한다. mysql 모듈을 설치하고 검증한다.

```
npm install mysql
```

```
// db-test.js
const mysql = require('mysql');
const connection = mysql.createConnection({
  host: '127.0.0.1',
  user: 'lab',
  password: '원하는비밀번호',
  database: 'lab'
});

connection.query('SELECT 1 + 1 AS result', (err, results) => {
  if (err) {
    console.error('연결 실패:', err);
    return;
  }
  console.log('연결 성공, 결과:', results[0].result);
  connection.end();
});
```

```
node db-test.js
```

쿼리 로그 활성화

2장에서는 "SQL 삽입 공격이 발생했을 때 서버가 실제로 어떤 쿼리를 실행했는가"를 눈으로 직접 보여주는 과정이 중요한 학습 포인트가 된다. 이를 위해 MySQL의 일반 쿼리 로그(general query log) 기능을 미리 켜 둔다.

```
SET GLOBAL general_log = 'ON';
SET GLOBAL general_log_file = '/var/log/mysql/general.log';
```

이 설정을 켜 두면 MySQL로 들어오는 모든 쿼리가 지정한 로그 파일에 순서대로 기록되며, 이후 `tail -f /var/log/mysql/general.log` 명령으로 실시간으로 들어오는 쿼리를 관찰할 수 있다. 다만 이 로그는 모든 쿼리를 기록하기 때문에 파일이 빠르게 커질 수 있어, 상시로 켜두기보다는 필요한 실습 시연 직전에 켜고 끝나면 끄는 방식으로 사용하는 것이 좋다.

백업·복구 기초

실습 중 데이터베이스의 상태가 의도치 않게 망가지는 경우(예를 들어 SQL 삽입 실습에서 `UPDATE` 문이 조건 없이 모든 행에 적용되어 데이터가 뒤섞이는 경우)를 대비해, 정상 상태의 스냅샷을 미리 저장해 두고 필요할 때 되돌릴 수 있도록 준비한다.

```
# 백업
mysqldump -u lab -p lab > lab_backup.sql

# 복구
mysql -u lab -p lab < lab_backup.sql
```

mysqldump는 데이터베이스의 현재 상태(테이블 구조와 데이터)를 SQL 문의 집합으로 추출해주는 도구다. 이 백업 파일을 시드 데이터 적재 직후, 즉 "실습이 시작되기 전 깨끗한 상태"에서 한 번 받아두면, 실습 중 데이터가 어떻게 망가지더라도 이 파일을 다시 적재하는 것만으로 원래 상태로 되돌릴 수 있다.

0.6 개발 도구 세팅

코드 편집기 선택과 원격 개발

이 교재의 실습 코드는 리눅스 서버 위에서 실행되지만, 코드를 작성하는 작업까지 서버에 직접 로그인해 터미널 편집기로 하는 것은 비효율적이다. Visual Studio Code(VS Code)의 Remote-SSH 확장 기능을 사용하면, 로컬 컴퓨터에서 익숙한 편집기 화면을 그대로 사용하면서 실제로는 SSH를 통해 원격 서버의 파일을 직접 열고 저장할 수 있다. 이 방식은 파일을 로컬로 내려받았다가 다시 서버로 올리는 번거로운 과정 없이, 마치 로컬 파일을 다루듯 서버의 코드를 편집할 수 있게 해준다.

주요 확장 도구

VS Code에서 이 교재의 실습에 유용한 확장 도구는 다음과 같다. ESLint는 자바스크립트 코드의 잠재적 오류나 스타일 문제를 편집 중에 바로 알려주는 정적 분석 도구다. Prettier는 코드의 들여쓰기나 따옴표 스타일 등을 자동으로 통일해주는 포맷터다. EJS 언어 지원 확장은 이 교재에서 뷰 템플릿으로 사용할 EJS 파일의 문법 강조를 도와준다. REST Client는 편집기 안에서 바로 HTTP 요청을 보내고 응답을 확인할 수 있게 해주는 확장으로, API를 테스트할 때 매번 별도의 프로그램을 열지 않아도 된다. DotENV는 환경변수를 저장하는 **.env** 파일의 문법을 인식해준다.

서버 내 최소 편집기 사용법

Remote-SSH 연결이 여의치 않은 상황이나, 서버에서 급히 설정 파일 하나를 고쳐야 하는 상황을 위해 터미널 기반 편집기의 최소한의 사용법도 익혀 둔다. **nano**는 화면 하단에 사용 가능한 단축키가 항상 표시되어 있어 입문자가 접근하기 쉬운 편집기이며, **vim**은 훨씬 강력하지만 처음 접하면 저장하고 나가는 방법조차 낯설 수 있다. **vim**을 처음 열었을 때는 **i** 키를 눌러 입력 모드로 전환한 뒤 내용을 수정하고, **Esc** 키로 입력 모드를 빠져나온 다음 **:wq**를 입력하고 엔터를 눌러 저장 후 종료한다. 이 최소한의 절차만 알아 두어도 서버 작업 중 급한 상황에서 당황하지 않을 수 있다.

Git 기본 설정과 .gitignore 구성

실습 코드는 Git으로 버전 관리하며, 처음 사용하는 서버에서는 사용자 정보를 등록해야 한다.

```
git config --global user.name "이름"
git config --global user.email "이메일"
```

프로젝트 디렉터리에는 버전 관리에서 제외할 파일과 디렉터리를 지정하는 **.gitignore** 파일을 둔다.

```
node_modules/
.env
uploads/
*.log
```

`node_modules`는 `package-lock.json`만 있으면 언제든지 다시 설치할 수 있으므로 저장소에 포함시킬 필요가 없다. `.env`는 데이터베이스 비밀번호 같은 민감한 설정값을 담는 파일인데, 이 파일이 실수로 Git 저장소에 커밋되어 공개 저장소에 그대로 노출되는 사고가 실무에서 실제로 자주 발생한다. `.gitignore`에 처음부터 등록해 두는 것이 이런 사고를 막는 가장 확실한 방법이다.

브라우저 개발자도구 활용

브라우저에 내장된 개발자도구는 이 교재 전체에서 취약점의 증상을 확인하는 주요 무대가 된다. Network 탭은 브라우저가 서버로 보내는 요청과 서버가 돌려주는 응답을 실시간으로 보여주며, 12장(CSRF)에서 토큰이 실제로 요청에 포함되는지 확인할 때, 13장(SSRF)에서 서버가 어떤 응답을 돌려받았는지 확인할 때 사용한다. Application 탭(크롬 기준)은 현재 페이지에 저장된 쿠키와 로컬 스토리지를 보여주는데, 14장(보안기능 결정에 사용되는 부적절한 입력값)에서 쿠키 값을 직접 조작해 권한을 위조하는 실습에서 핵심적으로 사용된다. Console 탭은 자바스크립트 코드를 직접 실행하고 오류 메시지를 확인할 수 있는 공간으로, 5장(XSS)에서 삽입된 스크립트가 실제로 실행되는지 확인할 때 사용한다.

API 테스트 도구

브라우저 주소창은 GET 요청만 손쉽게 보낼 수 있고, POST나 PUT 같은 다른 방식의 요청, 혹은 커스텀 헤더가 포함된 요청은 보내기 어렵다. `curl`은 명령줄에서 임의의 HTTP 요청을 자유롭게 구성해 보낼 수 있는 도구다.

```
curl -X POST http://localhost:3001/login \
  -H "Content-Type: application/json" \
  -d '{"username":"test","password":"1234"}'
```

Postman은 이런 요청을 그래픽 인터페이스로 구성하고, 자주 사용하는 요청을 저장해 재사용할 수 있게 해주는 프로그램이다. 두 도구 모두 이 교재에서 정상적인 요청과 조작된(공격) 요청을 각각 만들어 서버의 반응을 비교할 때 반복적으로 사용된다.

리눅스 점검 명령

실습 도중 서버의 상태를 확인해야 하는 상황이 자주 발생하므로, 다음 명령어들의 기본적인 사용법을 익혀 둔다.

`ss -tlnp`는 현재 서버에서 열려 있는 TCP 포트와 그 포트를 사용 중인 프로세스를 보여준다. 새 애플리케이션을 실행하려는데 "포트가 이미 사용 중"이라는 오류가 발생했을 때, 이 명령으로 어떤 프로세스가 해당 포트를 점유하고 있는지 확인할 수 있다.

```
ss -tlnp
```

`journalctl`은 `systemd`가 관리하는 서비스들의 로그를 조회하는 명령이다. 0.8절에서 각 애플리케이션을 `systemd` 서비스로 등록한 뒤에는, 이 명령으로 애플리케이션이 정상적으로 시작되었는지, 혹시 오류 메시지를 남기고 종료되었는지 확인하게 된다.

```
journalctl -u 서비스이름 -f    # -f는 실시간으로 새 로그를 계속 출력
```

ps aux는 현재 서버에서 실행 중인 모든 프로세스의 목록을 보여준다. **lsof**는 특정 파일이나 포트를 어떤 프로세스가 사용하고 있는지 확인하는 데 사용한다. **tail -f 파일경로**는 로그 파일의 마지막 부분을 실시간으로 계속 출력해주는 명령으로, 애플리케이션의 로그 파일을 실시간으로 관찰할 때 자주 사용한다.

웹 취약점 점검 도구

OWASP ZAP이나 Burp Suite Community Edition은 웹 애플리케이션에 자동화된 방식으로 취약점을 스캔하거나, 브라우저와 서버 사이를 오가는 요청을 가로채어 직접 변조해 보낼 수 있게 해주는 전문 도구다. 이 교재의 핵심 실습은 수작업으로 페이로드를 구성해 취약점의 원리를 이해하는 데 초점을 맞추지만, 이런 도구를 함께 소개해 두면 수강생이 실무에서 유사한 점검을 수행할 때 어떤 도구를 찾아야 하는지에 대한 실마리를 제공할 수 있다.

0.7 공통 애플리케이션 스캐폴드

"실제 운영 사이트처럼"의 기준

이 교재는 처음부터 "취약점을 보여주기 위한 최소한의 데모"가 아니라 "실제로 존재할 법한 사이트"를 만드는 것을 목표로 삼았다. 이 기준을 구체적인 코드 구조로 옮기면 다음과 같다. 첫째, 화면은 헤더·내비게이션·사이드바·푸터를 갖춘 하나의 일관된 레이아웃 위에서 구성된다. 둘째, 로그인하지 않은 사용자와 로그인한 사용자가 볼 수 있는 화면이 구분된다. 셋째, 정상적이지 않은 요청(존재하지 않는 페이지, 서버 오류)에 대해 사용자에게 의미 있는 오류 화면을 보여준다. 넷째, 모든 요청은 로그로 남는다. 이 네 가지 조건을 매번 새로 구현하지 않도록, 이번 절에서 13개 애플리케이션이 공통으로 사용할 뼈대를 한 번만 만들어 두고 이후 각 장에서는 이 뼈대를 복제해 그날의 취약점 기능만 추가한다.

표준 디렉터리 구조

```
lab01-sqli/
├── app.js                # 애플리케이션 진입점
├── package.json
├── .env                  # 환경변수 (Git에 포함하지 않음)
├── config/
│   └── index.js          # 설정값 로딩
├── models/
│   ├── db.js             # mysql 드라이버 커넥션 풀
│   └── sequelize.js      # Sequelize 인스턴스
├── routes/
│   ├── index.js          # 공통 라우터 (대시보드 등)
│   ├── vuln.js           # 취약 버전 라우트
│   └── patched.js        # 안전 버전 라우트
├── controllers/
│   └── ...                # 라우트가 호출하는 실제 처리 로직
├── services/
│   └── ...                # 데이터베이스 접근 등 비즈니스 로직
├── middlewares/
│   ├── auth.js           # 로그인 여부 확인
│   └── errorHandler.js   # 공통 에러 처리
├── views/
│   └── layout.ejs        # 공통 레이아웃
```

```

├── partials/
│   ├── header.ejs
│   ├── nav.ejs
│   └── footer.ejs
├── index.ejs
├── error.ejs
└── public/
    ├── css/style.css
    └── js/main.js

```

이 구조에서 **routes**는 URL 경로와 처리 함수를 연결하는 역할만 하고, 실제 로직은 **controllers**와 **services**로 분리한다. 이렇게 계층을 나누는 이유는 코드를 정돈하기 위한 목적도 있지만, 이후 각 장에서 "취약한 버전"과 "안전한 버전"을 나란히 두고 비교할 때 두 버전의 차이가 정확히 어느 계층에 있는지 명확하게 드러나기 때문이다.

디렉터리 생성

```
mkdir -p config models routes controllers services middlewares views/partial
public/css public/js
```

파일 생성

```
touch app.js .env
touch config/index.js
touch models/db.js models/sequelize.js
touch routes/index.js routes/vuln.js routes/patched.js
touch middlewares/auth.js middlewares/errorHandler.js
touch views/layout.ejs views/index.ejs views/error.ejs
touch views/partial/header.ejs views/partial/nav.ejs views/partial/footer.ejs
touch public/css/style.css public/js/main.js
```

Express 애플리케이션 기본 구성

app.js는 다음과 같은 형태로 시작한다.

```
require('dotenv').config();
const express = require('express');
const path = require('path');
const morgan = require('morgan');
const fs = require('fs');

const config = require('./config');
const indexRouter = require('./routes/index');
const vulnRouter = require('./routes/vuln');
const patchedRouter = require('./routes/patched');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
```

```

// 뷰 엔진 설정
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// 요청 본문 파싱
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// 정적 파일 서빙
app.use('/static', express.static(path.join(__dirname, 'public')));

// 요청 로깅 (파일로 기록)
const accessLogStream = fs.createWriteStream(
  path.join('/var/log/labs', 'lab01-access.log'),
  { flags: 'a' }
);
app.use(morgan('combined', { stream: accessLogStream }));

// 라우터 연결
app.use('/', indexRouter);
app.use('/lab/vuln', vulnRouter);
app.use('/lab/patched', patchedRouter);

// 헬스체크
app.get('/healthz', (req, res) => res.status(200).send('ok'));

// 404 처리
app.use((req, res) => {
  res.status(404).render('error', { title: '페이지를 찾을 수 없습니다', status: 404 });
});

// 공통 에러 핸들러
app.use(errorHandler);

const PORT = config.port || 3001;
app.listen(PORT, () => {
  console.log(`lab01-sqli 서버가 ${PORT}번 포트에서 실행 중입니다.`);
});

```

이 코드에서 눈여겨볼 지점은 라우터를 `/lab/vuln`과 `/lab/patched`라는 두 개의 경로로 분리해 둔 부분이다. 이렇게 해두면 같은 애플리케이션 안에서 "이 기능의 취약한 버전"과 "패치된 버전"을 URL 하나만 바꿔서 즉시 오갈 수 있어, 강의 중 두 코드의 동작 차이를 그 자리에서 바로 비교해 보여줄 수 있다.

필요한 npm 패키지 설치

현재 app.js에서 사용하는 외부 패키지는 다음과 같다.

- express: Node.js로 웹 서버를 만들 때 사용하는 핵심 도구
- dotenv: 비밀번호, DB 정보, 포트 등의 설정값을 코드와 분리하는 도구
- morgan: 누가 언제 어떤 URL로 접속했는지 기록하는 도구

- ejs: JavaScript 데이터를 HTML 화면에 넣어서 동적으로 웹 페이지를 만드는 도구

```
npm install express dotenv morgan ejs
```

필요한 디렉터리 생성

app.js에서 다음 경로를 사용한다.

```
require('./routes/index');  
require('./routes/vuln');  
require('./routes/patched');  
require('./middlewares/errorHandler');
```

따라서 먼저 디렉터리를 만들어야 한다.

```
mkdir -p routes  
mkdir -p middlewares  
mkdir -p views  
mkdir -p public
```

config.js 생성

app.js에서 `const config = require('./config');`를 사용하고 있으므로 프로젝트 루트에 config.js가 반드시 있어야 한다.

.env 파일 생성

첫 번째 줄에서 `require('dotenv').config();`을 사용한다.

```
touch .env
```

7. 라우터 파일 생성

app.js는 다음 세 개의 라우터를 불러온다.

```
const indexRouter = require('./routes/index');  
const vulnRouter = require('./routes/vuln');  
const patchedRouter = require('./routes/patched');
```

따라서 최소한 다음 파일들이 필요하다.

```
touch routes/index.js
touch routes/vuln.js
touch routes/patched.js
```

에러 핸들러 생성

app.js에서 `const errorHandler = require('./middlewares/errorHandler');`를 사용하고 있다. 따라서 다음 파일이 필요하다.

```
touch middlewares/errorHandler.js
```

EJS 에러 페이지 생성

app.js에서 404 처리 시 아래 코드를 사용한다.

```
res.status(404).render('error', {
  title: '페이지를 찾을 수 없습니다',
  status: 404
});
```

그리고 에러 핸들러에서도 아래 코드를 사용한다.

```
res.status(...).render('error', ...)
```

따라서 다음 파일이 필요하다.

```
touch views/error.ejs
```

로그 디렉터리 생성

app.js에서 가장 주의해야 하는 부분이다.

```
const accessLogStream = fs.createWriteStream(
  path.join('/var/log/labs', 'lab01-access.log'),
  { flags: 'a' }
);
```

즉, 애플리케이션 실행 전에 다음 디렉터리가 존재해야 한다.

```
sudo mkdir -p /var/log/labs
```

그런데 여기서 권한 문제가 발생할 수 있다. 현재 사용자(labuser)로 Node.js를 실행한다면 /var/log/labs에 로그를 쓸 수 있어야 한다.

```
sudo chown -R labuser:labuser /var/log/labs
```

public 디렉터리 준비

다음 코드가 있다.

```
app.use('/static', express.static(path.join(__dirname, 'public')));
```

따라서 디렉터리가 필요하다.

```
mkdir public
```

정적 파일 서빙과 에러 핸들러

`express.static` 미들웨어는 CSS나 자바스크립트, 이미지 같은 정적 파일을 별도의 라우트 코드 없이 자동으로 제공해주는 기능이다. 위 예제에서 `/static` 경로 뒤에 오는 요청은 `public` 디렉터리 안의 실제 파일과 매칭되어 그대로 응답된다.

공통 에러 핸들러(`middlewares/errorHandler.js`)는 애플리케이션 코드 어딘가에서 예외가 발생했을 때 이를 최종적으로 받아 처리하는 역할을 한다.

```
// middlewares/errorHandler.js
module.exports = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).render('error', {
    title: '서버 오류가 발생했습니다',
    status: 500
  });
};
```

이 핸들러가 사용자에게 서버 내부의 오류 메시지나 스택 트레이스를 그대로 노출하지 않고 일반화된 문구만 보여준다는 점은, 이 교재 전반에서 "에러 메시지 정보노출"을 막는 원칙으로 반복해서 언급된다.

공통 레이아웃 구성

views/layout.ejs는 모든 페이지가 공유하는 뼈대다. EJS는 HTML 안에 자바스크립트 코드를 삽입해 동적으로 화면을 렌더링할 수 있게 해주는 템플릿 엔진이다.

```
<!-- views/layout.ejs -->
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title><%= title %></title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <%- include('partials/header') %>
  <div class="layout">
    <%- include('partials/nav') %>
    <main class="content">
      <%- body %>
    </main>
  </div>
  <%- include('partials/footer') %>
</body>
</html>
```

partials/header.ejs, **partials/nav.ejs**, **partials/footer.ejs**는 각각 상단 헤더, 좌측(또는 상단) 내비게이션 메뉴, 하단 푸터를 담당하는 조각으로 분리해 두어, 여러 페이지에서 반복되는 부분을 한 곳에서만 관리할 수 있게 한다.

전체 구조

```
layout.ejs
├── <head>
│   ├── 문자 인코딩
│   ├── 페이지 제목
│   └── CSS
├── header.ejs
├── layout
│   ├── nav.ejs
│   └── body (페이지별 내용)
└── footer.ejs
```

즉, 웹 페이지를 다음과 같이 구성하는 것이다.



이 문서는 HTML5 문서라는 것을 브라우저에 알려주는 선언이다. 브라우저가 HTML을 올바른 표준 방식으로 해석하도록 한다.

HTML 문서의 시작을 의미한다. lang="ko"는 이 페이지의 기본 언어가 한국어라는 뜻이다. `<html lang="en">`이라면 영어 페이지라는 의미이다. 이 정보는 브라우저나 검색엔진, 스크린 리더 등이 사용할 수 있다.

사용자 화면에 직접 표시되는 본문이 아니라, 페이지에 대한 설정 정보를 넣는 영역이다. 현재 코드에서는 다음 세 가지가 들어 있다.

```

<head>
|
├─ 문자 인코딩
├─ 페이지 제목
└─ CSS
</head>
  
```

HTML 문서의 문자 인코딩을 UTF-8로 지정한다. 한국어를 포함한 다양한 문자를 정상적으로 표시하기 위해 사용한다. 예를 들어 이 설정이 없거나 잘못되면 안녕하세요가 `ìëìì,ì`와 같이 깨질 수도 있다. 따라서 한국어 웹 페이지에서는 일반적으로 UTF-8을 사용한다.

`<title><%= title %></title>`

여기서 중요한 부분은 `<%= title %>`이다. 이것은 EJS 문법이다. title이라는 JavaScript 변수를 HTML에 출력한다는 의미이다. 예를 들어 Express에서

```

res.render('error', {
  title: '페이지를 찾을 수 없습니다'
});
  
```

라고 전달하면

```
<title><%= title %></title>
```

가 실제 브라우저에서는

```
<title>페이지를 찾을 수 없습니다</title>
```

로 변환된다.

브라우저 탭에는 페이지를 찾을 수 없습니다라고 표시된다.

CSS 연결 ()

웹 페이지의 디자인을 담당하는 CSS 파일을 연결한다. 여기서 `/static/css/style.css`라는 URL 요청을 하게 된다. 앞에서 작성한 `app.js`에는 `app.use('/static', express.static(path.join(__dirname, 'public')));`가 있었다. 이 코드를 기준으로 보면

```
브라우저 요청
/static/css/style.css
|
▼
Express
|
▼
public/css/style.css
```

로 연결된다. 즉, `/static/css/style.css`는 실제 파일 `public/css/style.css`를 의미한다.

Header 포함 (<%- include('partials/header') %>)

이 부분은 다른 EJS 파일을 현재 위치에 삽입(include)하는 코드이다. 즉,

```
views/
├─ layout.ejs
└─ partials/
   └─ header.ejs
```

라는 구조를 예상할 수 있다. `header.ejs`가

```
<header>
  <h1>Lab 01 - SQL Injection</h1>
</header>
```

라면 `<%- include('partials/header') %>` 부분이 실제 HTML로 합쳐져서 결과적으로

```
<header>
  <h1>Lab 01 - SQL Injection</h1>
</header>
```

가 된다. 이렇게 공통 영역을 별도 파일로 분리하면 편리하다. 예를 들어 모든 페이지에서 Header가 동일하다면 `index.ejs`, `vuln.ejs`, `patched.ejs` 각각에 Header 코드를 반복해서 작성할 필요가 없다.

웹 페이지의 주요 콘텐츠 영역을 감싸는

이다. `class="layout"`을 CSS에서 사용해서 화면 배치를 지정할 수 있다. 예를 들어 CSS에서

```
.layout {
  display: flex;
}
```

라고 하면 내부의 `nav`와 `main`을 가로로 배치할 수 있다.

Navigation 포함 (<%- include('partials/nav') %>)

Header와 마찬가지로 별도의 EJS 파일을 포함한다.

구조

```
views/
├── layout.ejs
└── partials/
    ├── header.ejs
    ├── nav.ejs
    └── footer.ejs
```

`nav.ejs`가

```
<nav>
  <ul>
    <li><a href="/">홈</a></li>
    <li><a href="/lab/vuln">취약점 실습</a></li>
    <li><a href="/lab/patched">보안 적용</a></li>
  </ul>
</nav>
```

라면 해당 HTML이 `layout.ejs`에 삽입된다.

여기는 실제 페이지별 콘텐츠가 들어가는 영역이다. 예를 들어,

```

홈 페이지
↓
홈 화면 내용

취약점 실습 페이지
↓
SQL Injection 실습 내용

보안 적용 페이지
↓
Prepared Statement 적용 내용

```

처럼 페이지마다 내용이 달라질 수 있다.

<%- body %>

이 부분이 이 레이아웃에서 가장 핵심적인 부분이다. body 변수에 들어있는 HTML을 현재 위치에 삽입한다. 예를 들어 페이지의 body가

```

<h2>SQL Injection 실습</h2>
<p>취약한 SQL 쿼리를 테스트합니다.</p>

```

라면 <%- body %> 자리에 들어가서 최종적으로

```

<main class="content">
  <h2>SQL Injection 실습</h2>
  <p>취약한 SQL 쿼리를 테스트합니다.</p>
</main>

```

가 된다. 따라서 layout.ejs는 공통 구조를 담당하고, body는 페이지별 콘텐츠를 담당한다.

Footer 포함 (<%- include('partials/footer') %>)

Footer도 공통 컴포넌트로 분리되어 있다. 예를 들어,

```

<footer>
  <p>Lab 01</p>
</footer>

```

가 들어갈 수 있다.

최종적으로 브라우저에 전달되는 모습

예를 들어 다음 파일들이 있다고 가정한다.

```
views/
├── layout.ejs
├── partials/
│   ├── header.ejs
│   ├── nav.ejs
│   └── footer.ejs
└── vuln.ejs
```

그리고

header.ejs

```
<header>
  <h1>SQL Injection Lab</h1>
</header>
```

nav.ejs

```
<nav>
  <a href="/">홈</a>
  <a href="/lab/vuln">취약점 실습</a>
  <a href="/lab/patched">보안 적용</a>
</nav>
```

footer.ejs

```
<footer>
  <p>Copyright 2026</p>
</footer>
```

vuln.ejs

```
<h2>SQL Injection 취약점 실습</h2>
<p>취약한 SQL 쿼리를 확인합니다.</p>
```

최종적으로 브라우저에 전달되는 HTML은 개념적으로 아래와 같다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>SQL Injection Lab</title>
```

```

<link rel="stylesheet" href="/static/css/style.css">
</head>

<body>

  <header>
    <h1>SQL Injection Lab</h1>
  </header>

  <div class="layout">

    <nav>
      <a href="/">홈</a>
      <a href="/lab/vuln">취약점 실습</a>
      <a href="/lab/patched">보안 적용</a>
    </nav>

    <main class="content">
      <h2>SQL Injection 취약점 실습</h2>
      <p>취약한 SQL 쿼리를 확인합니다.</p>
    </main>

  </div>

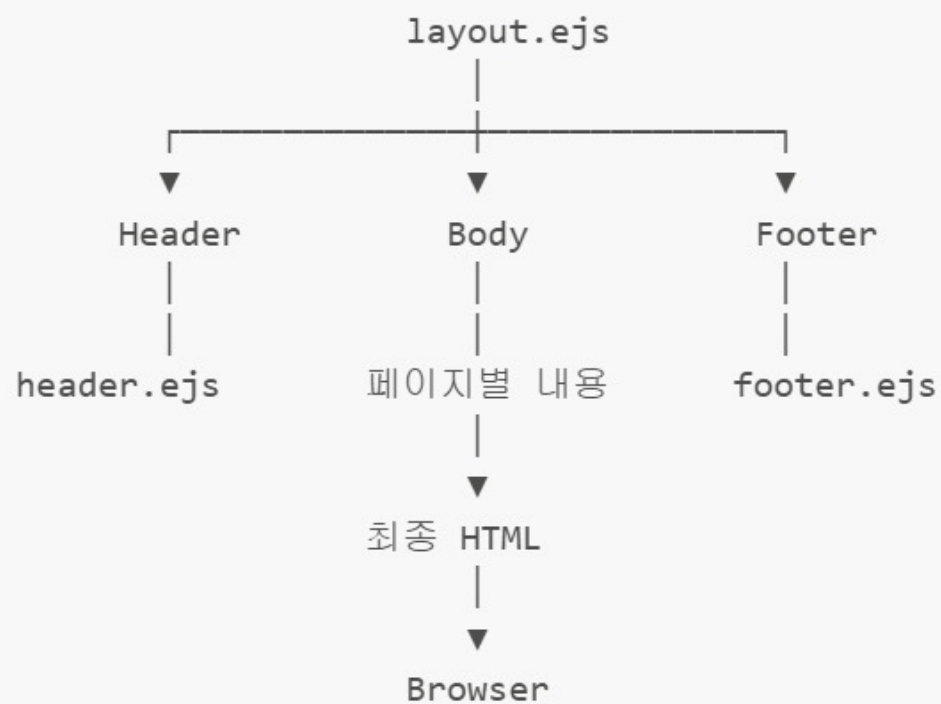
  <footer>
    <p>Copyright 2026</p>
  </footer>

</body>
</html>

```

핵심 정리

이 파일의 역할은 웹 페이지의 공통 틀을 만드는 것이다.



특히 EJS의 문법은 두 가지를 기억하면 좋다.

```
<%= 변수 %>
```

→ 변수 값을 HTML에 출력

```
<%- include('파일') %>
```

→ 다른 EJS 파일을 현재 위치에 삽입

```
<%- body %>
```

→ 페이지별 본문을 레이아웃 안에 삽입

세션 기반 로그인 골격

로그인 기능은 대부분의 실습 애플리케이션에서 공통으로 필요하므로, `express-session`을 이용한 기본 골격을 미리 만들어 둔다.

```
// app.js에 추가
const session = require('express-session');

app.use(session({
  secret: config.sessionSecret,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, maxAge: 1000 * 60 * 60 } // 1시간
}));
```

```
// middlewares/auth.js
module.exports = function requireLogin(req, res, next) {
  if (!req.session.user) {
    return res.redirect('/login');
  }
  next();
};
```

이 `requireLogin` 미들웨어를 로그인에 필요한 라우트 앞에 붙여 두면, 세션에 로그인 정보가 없는 요청은 자동으로 로그인 페이지로 돌려보내진다. 이 골격은 14장에서 "쿠키가 아닌 세션에 관한 정보를 저장해야 하는 이유"를 실감하는 기반이 된다.

전체적인 흐름

```
// app.js에 추가
const session = require('express-session');

app.use(session({
  secret: config.sessionSecret,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, maxAge: 1000 * 60 * 60 } // 1시간
}));
```

이 코드는 Express 애플리케이션에 세션(Session) 기능을 추가하는 코드이다. 쉽게 말하면, 사용자가 웹사이트에 로그인했을 때 서버가 "이 사용자는 로그인한 사용자다"라는 정보를 기억할 수 있도록 만드는 역할을 한다. 전체적인 흐름은 다음과 같다.



express-session 가져오기 (const session = require('express-session'))

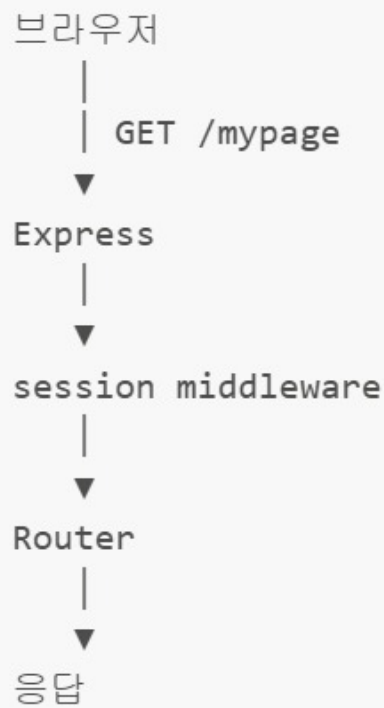
express-session이라는 패키지를 가져오는 코드이다. express-session은 Express에서 세션을 관리할 수 있게 해주는 미들웨어이다. 따라서 먼저 설치해야 한다.

```
npm install express-session
```

app.use(session(...))

Express 애플리케이션에 세션 미들웨어를 등록하는 코드이다. **app.use()**는 Express에서 미들웨어를 등록할 때 사용하는 함수이다. 즉, 사용자가 서버에 요청을 보낼 때마다 세션 미들웨어가 요청을 처리한다. 예를 들어, 이런 흐름으로 동

작한다.



secret (secret: config.sessionSecret,)

세션을 보호하기 위한 비밀 키(secret key)를 설정한다. 여기서는 config.sessionSecret에 저장된 값을 사용한다. 예를 들어 config.js가

```

module.exports = {
  port: 3001,
  sessionSecret: 'my-secret-key'
};

```

라면 `secret: config.sessionSecret`은 사실상 `secret: 'my-secret-key'`와 같은 의미이다. 다만 실제 프로젝트에서는 비밀 키를 코드에 직접 작성하는 것보다 .env에 저장하는 것이 좋다.

예

```
SESSION_SECRET=very-long-random-secret-key
```

그리고

```

module.exports = {
  port: process.env.PORT || 3001,
  sessionSecret: process.env.SESSION_SECRET
};

```

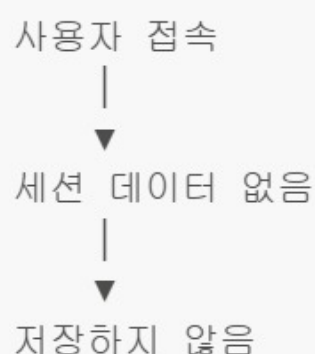
이런 식으로 관리할 수 있다.

resave: false

세션 데이터가 변경되지 않았더라도 매 요청마다 세션을 다시 저장할 것인지 설정한다. `resave: false`는 "세션 내용이 변경되지 않았다면 굳이 다시 저장하지 않는다."라는 의미이다. 일반적으로 불필요한 세션 저장을 줄이기 위해 `false`를 사용하는 경우가 많다.

saveUninitialized: false

아직 아무 데이터도 저장하지 않은 세션을 서버에 저장할 것인지 설정한다. 예를 들어 사용자가 웹사이트에 접속했지만 `req.session.user = ...`와 같이 세션 데이터를 실제로 사용하지 않았다면 세션을 저장하지 않는다. 즉,



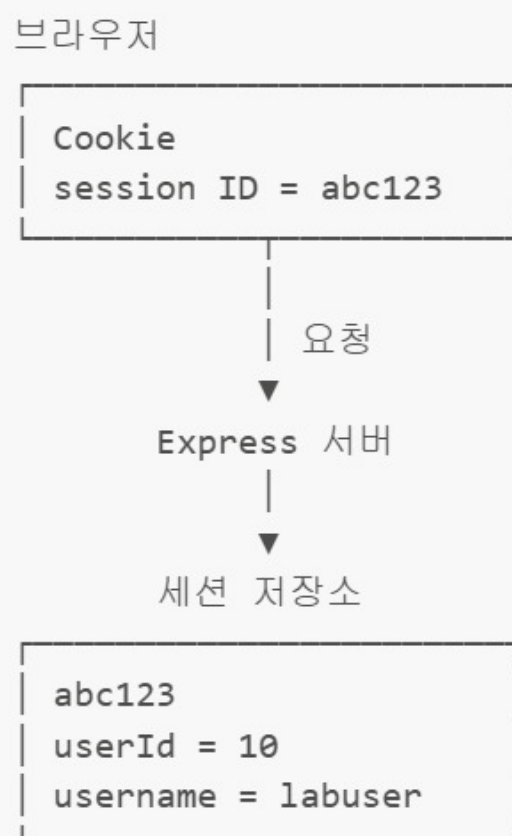
이라는 의미이다. 반대로 `req.session.userId = 10;`처럼 세션을 사용하면 세션 데이터가 저장된다.

cookie

```

cookie: {
  httpOnly: true,
  maxAge: 1000 * 60 * 60
}
  
```

브라우저에 저장되는 세션 쿠키의 설정이다. 여기서 중요한 것은 세션 데이터 자체와 세션 쿠키는 다르다는 점이다. 일반적인 세션 구조는 아래와 같다.



브라우저에는 보통 세션을 식별하기 위한 쿠키가 저장되고, 실제 세션 데이터는 서버 측에 저장된다.

httpOnly: true

이 옵션은 세션 쿠키를 JavaScript 코드에서 직접 읽지 못하도록 한다. 예를 들어 브라우저에서 `document.cookie`를 실행해도 HttpOnly가 설정된 쿠키는 JavaScript로 접근할 수 없다. 따라서 다음과 같은 공격에 대한 방어에 도움이 된다.



다만 HttpOnly가 XSS 자체를 막는 것은 아니다. XSS 공격이 발생하면 공격자가 페이지 내에서 다른 악성 동작을 수행할 수 있으므로 XSS 방어는 별도로 필요하다.

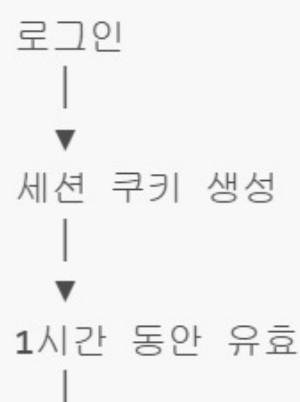
maxAge

`maxAge: 1000 * 60 * 60`

쿠키가 유지되는 시간을 설정한다. 단위는 밀리초(ms)이다. 계산해 보면

$1000 \text{ ms} = 1\text{초}$
 $60 \times 1000 \text{ ms} = 1\text{분}$
 $60 \times 60 \times 1000 \text{ ms} = 1\text{시간}$

따라서 `maxAge: 1000 * 60 * 60`은 "세션 쿠키의 유효 기간을 1시간으로 설정"한다는 의미이다. 즉,



▼
1시간 후 만료

이다.

전체 코드를 해석하면

```
const session = require('express-session');

app.use(session({
  secret: config.sessionSecret,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    maxAge: 1000 * 60 * 60
  }
}));
```

전체적으로 다음과 같은 의미이다.

Express 서버에서 세션 기능을 활성화하고, `config.sessionSecret`을 사용해 세션을 보호하며, 변경되지 않은 세션은 불필요하게 저장하지 않고, 초기화되지 않은 세션도 저장하지 않는다. 또한 세션 쿠키는 JavaScript에서 접근할 수 없도록 하고 1시간 동안 유지한다.

실습 환경에서 꼭 알아둘 점

현재 코드처럼 `express-session`의 기본 설정을 사용하면 세션 저장소를 별도로 지정하지 않았기 때문에 기본적으로 `MemoryStore`를 사용한다. 즉,

```
app.use(session({
  ...
}));
```

만 사용하는 실습 환경에서는 세션 데이터가 Node.js 프로세스의 메모리에 저장된다.

따라서

- 서버를 재시작하면 세션이 사라질 수 있음
- 여러 서버로 확장하기 어려움
- 운영 환경에서는 권장되지 않음

실제 운영 환경이라면 Redis, DB 기반 세션 저장소 등을 사용하는 것이 일반적이다. 또한 `cookie.maxAge`가 1시간이라고 해서 반드시 서버 측 세션도 정확히 1시간 후 삭제된다는 의미는 아닙니다. 쿠키의 만료 시간과 서버의 세션 저장소에서 세션을 제거하는 정책은 별개의 문제이다.

한 줄 요약

`express-session`
= "사용자가 누구인지 서버가 요청 간에 기억할 수 있도록 해주는 Express용 세션 관리 미들웨어"

전체적인 흐름

```
// middlewares/auth.js
module.exports = function requireLogin(req, res, next) {
  if (!req.session.user) {
    return res.redirect('/login');
  }
  next();
};
```

이 코드는 로그인한 사용자만 특정 페이지에 접근할 수 있도록 제한하는 인증 미들웨어(Authentication Middleware)이다. 파일 위치가 `middlewares/auth.js`이고, 이름이 `requireLogin`인 것으로 보아 로그인이 필요한 페이지를 보호하는 역할을 한다. 전체적인 동작은 다음과 같다.



하나씩 살펴본다.

module.exports (module.exports = function requireLogin(req, res, next) {})

이 코드는 `requireLogin`이라는 함수를 다른 파일에서 사용할 수 있도록 내보내는 것이다. 즉, 다른 라우터에서 `const requireLogin = require('../middlewares/auth');`처럼 가져올 수 있다.

req, res, next

Express 미들웨어 함수는 일반적으로 다음과 같은 형태이다.

```
function (req, res, next) {
}
```

각각의 의미는 다음과 같다.

- req: 사용자의 요청(Request) 정보를 담고 있다.
- 예를 들어 req.session을 통해 현재 사용자의 세션 정보에 접근할 수 있다.
- res: 서버가 사용자에게 응답(Response)을 보내기 위해 사용한다.
- 예: res.send('Hello'); 또는 res.redirect('/login');
- next: 현재 미들웨어의 작업이 끝났으니 다음 미들웨어 또는 다음 라우터로 이동하라는 의미이다.
- 예: next();를 호출하면 다음 단계로 넘어간다.

설정값 분리

데이터베이스 비밀번호나 세션 암호화 키 같은 민감한 값은 코드에 직접 적어 넣지 않고 환경변수 파일(.env)에 분리해 관리한다.

```
# .env
PORT=3001
DB_HOST=127.0.0.1
DB_USER=lab
DB_PASSWORD=원하는비밀번호
DB_NAME=lab
SESSION_SECRET=아무도_모를_긴_무작위_문자열
```

```
// config/index.js
require('dotenv').config();

module.exports = {
  port: process.env.PORT,
  db: {
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME
  },
  sessionSecret: process.env.SESSION_SECRET
};
```

이렇게 분리해 두면 .env 파일을 .gitignore에 등록해 저장소에는 절대 포함되지 않도록 할 수 있고, 서버마다 다른 값(비밀번호, 포트 등)을 코드 수정 없이 파일 하나만 바꿔서 적용할 수 있다.

요청 로깅

앞선 `app.js` 예제에서 `morgan` 미들웨어를 이용해 모든 HTTP 요청을 `/var/log/labs` 아래 로그 파일로 기록하도록 설정했다. 이 로그는 이후 여러 장에서 "공격이 실제로 어떤 요청 형태로 들어왔는가"를 사후에 다시 살펴볼 때 유용하게 쓰인다.

헬스체크 엔드포인트

`/healthz`라는 이름의 간단한 엔드포인트를 모든 애플리케이션에 공통으로 둔다. 이 엔드포인트는 단순히 "ok"라는 문자열만 응답하며, 애플리케이션이 정상적으로 살아서 요청을 받을 수 있는 상태인지를 빠르게 확인하는 용도로 사용한다. 0.9절의 환경 점검과 0.8절의 서비스 등록 이후 상태 확인 과정에서 이 엔드포인트를 반복적으로 사용하게 된다.

앱 공통 페이지 구조

각 실습 애플리케이션은 `/lab/vuln`, `/lab/patched` 외에도 `/lab/report`라는 페이지를 공통으로 둔다. 이 페이지는 그 장에서 다루는 취약점의 개요, 취약한 코드와 안전한 코드의 핵심 차이, 참고자료 링크를 한 화면에 정리해 보여주는 요약 페이지로, 강의 말미에 그 장의 내용을 정리할 때 화면 하나로 복습할 수 있게 해준다.

취약/안전 라우트 공존 설계

이 스캐폴드의 가장 중요한 설계 결정은 "취약한 코드와 안전한 코드를 별도의 프로젝트로 나누지 않고, 하나의 애플리케이션 안에 라우트만 다르게 하여 공존시킨다"는 점이다. 만약 두 버전을 완전히 별개의 프로젝트로 만든다면, 강의 중 두 코드를 비교하기 위해 서버를 켜다 켜거나 포트를 바꿔가며 오가야 하는 번거로움이 생긴다. 반면 하나의 애플리케이션 안에 `/lab/vuln/...`과 `/lab/patched/...`라는 두 경로로 공존시켜 두면, 브라우저 주소창의 경로 한 부분만 바뀌서 즉시 두 버전의 동작 차이를 나란히 시연할 수 있다. 이 설계는 이후 모든 장에서 반복적으로 활용된다.

0.8 서비스 운영 — systemd와 Nginx

상시 실행 서비스가 필요한 이유

지금까지의 과정으로 터미널에서 `node app.js`를 실행하면 애플리케이션이 동작하는 상태까지는 만들 수 있다. 하지만 이 방식에는 명백한 한계가 있다. 터미널 창을 닫으면 애플리케이션도 함께 종료되고, 서버가 재부팅되면 모든 애플리케이션을 하나하나 다시 실행해야 하며, 애플리케이션이 예기치 못한 오류로 죽어버려도 이를 자동으로 다시 살려주는 장치가 없다. 실습 서버가 강의 내내 13개의 애플리케이션을 안정적으로 서비스하려면, 이 애플리케이션들을 운영체제 차원에서 관리되는 "서비스"로 등록해야 한다. 이 역할을 담당하는 것이 리눅스의 서비스 관리자인 `systemd`다.

systemd 유닛 파일 구조

`systemd`는 관리할 서비스마다 "유닛 파일"이라는 설정 파일을 요구한다. `lab01-sqli` 애플리케이션의 유닛 파일은 `/etc/systemd/system/lab01.service` 경로에 다음과 같이 작성한다.

```
[Unit]
Description=lab01-sqli - SQL 삽입 실습 애플리케이션
After=network.target mysql.service

[Service]
Type=simple
User=labuser
WorkingDirectory=/opt/labs/lab01-sqli
ExecStart=/usr/bin/node /opt/labs/lab01-sqli/app.js
```

```
EnvironmentFile=/opt/labs/lab01-sqli/.env
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

[Unit] 영역의 **After**는 이 서비스가 네트워크와 MySQL 서비스가 준비된 이후에 시작되어야 함을 지정한다.

[Service] 영역의 **User=labuser**는 이 프로세스가 관리자 계정이 아닌, 0.2절에서 만든 권한이 제한된 실습 계정으로 실행되도록 지정하는 부분으로, 앞서 강조한 최소 권한 원칙이 여기서 실제로 구현된다. **WorkingDirectory**와 **ExecStart**는 각각 이 프로세스가 실행될 디렉터리와 실제로 실행할 명령을 지정한다. **EnvironmentFile**은 **.env** 파일에 정의된 환경변수를 프로세스 시작 시 함께 불러오도록 지정한다. **Restart=always**는 이 프로세스가 어떤 이유로든 종료되면 **systemd**가 자동으로 다시 시작해주도록 하는 설정으로, 이는 서비스의 안정성을 크게 높여준다.

서비스 등록·기동·상태 확인·재시작

유닛 파일을 새로 만들거나 수정했다면, **systemd**에게 이 변경을 알려주는 절차가 필요하다.

```
sudo systemctl daemon-reload
sudo systemctl enable --now lab01
sudo systemctl status lab01
```

daemon-reload는 **systemd**가 유닛 파일들을 다시 읽어 들이도록 하는 명령이며, 유닛 파일을 새로 추가하거나 내용을 바꿀 때마다 반드시 먼저 실행해야 한다. **enable --now**는 앞서와 마찬가지로 즉시 시작과 부팅 시 자동 시작을 함께 설정한다. 이후 애플리케이션 코드를 수정했다면 다음 명령으로 서비스를 재시작해 변경사항을 반영한다.

```
sudo systemctl restart lab01
```

로그 확인

systemd가 관리하는 서비스의 표준 출력(콘솔에 찍히는 **console.log** 내용 등)은 자동으로 **journalctl**을 통해 조회할 수 있다.

```
journalctl -u lab01 -f
```

이 명령을 실행해 두면, 애플리케이션이 시작될 때 남기는 메시지나 실행 중 발생하는 오류를 실시간으로 지켜볼 수 있다. 애플리케이션이 예상대로 동작하지 않을 때 가장 먼저 확인해야 할 곳이 바로 이 로그다.

다중 앱 관리를 위한 유닛 템플릿

13개의 애플리케이션마다 매번 유닛 파일을 처음부터 작성하는 것은 비효율적이다. **systemd**는 **@** 기호를 이용한 템플릿 유닛 기능을 제공해, 하나의 템플릿으로 여러 개의 유사한 서비스를 관리할 수 있게 해준다. 예를 들어 **/etc/systemd/system/lab@.service**라는 템플릿을 만들어 두면, **lab01**, **lab02**처럼 이름 뒤에 붙는 식별자에 따라 각기 다른 디렉터리의 애플리케이션을 같은 방식으로 실행할 수 있다. 다만 각 애플리케이션의 포트 번호와 디렉터리

가 서로 다르므로, 이 교재에서는 이해를 명확히 하기 위해 각 장마다 개별 유닛 파일을 만드는 방식을 기본으로 설명하고, 템플릿 방식은 관리 효율을 높이고 싶을 때 선택적으로 적용할 수 있는 심화 방법으로 다룬다.

Nginx란, 리버스 프록시 개념

지금까지의 설정으로 각 애플리케이션은 3001번부터 3013번까지의 포트에서 개별적으로 실행되고 있다. 하지만 실제 웹사이트는 사용자가 포트 번호를 일일이 기억해서 접속하지 않고, `http://사이트주소`처럼 포트 번호 없이(즉 기본 포트인 80번으로) 접속하는 것이 일반적이다. 또한 여러 애플리케이션을 하나의 도메인 아래 각기 다른 경로나 서브도메인으로 자연스럽게 연결해주는 역할도 필요하다. 이런 역할을 담당하는 것이 리버스 프록시이며, 이 교재에서는 Nginx를 사용한다.

리버스 프록시는 사용자의 요청을 가장 먼저 받아, 그 요청의 내용(경로나 도메인)을 살펴본 뒤 적절한 뒷단(백엔드) 서버로 전달해주는 역할을 한다. 사용자 입장에서는 Nginx 하나에만 접속하는 것처럼 보이지만, 실제로는 Nginx가 그 뒤에서 요청의 성격에 맞는 Node.js 애플리케이션으로 요청을 중계해준다.

Nginx 설치와 프록시 설정

```
sudo apt install -y nginx      # Rocky: sudo dnf install -y nginx
sudo systemctl enable --now nginx
```

`lab01-sqli` 애플리케이션을 위한 Nginx 설정은 `/etc/nginx/sites-available/lab01.conf`에 다음과 같이 작성한다(Rocky 계열은 `/etc/nginx/conf.d/lab01.conf`).

```
server {
    listen 80;
    server_name lab01.local;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

`proxy_pass`는 이 설정 블록으로 들어온 모든 요청을 `127.0.0.1:3001`, 즉 `lab01-sqli` 애플리케이션으로 전달하라는 지시다. 그 아래 `proxy_set_header` 항목들은 원래 요청에 담겨 있던 정보(실제 접속자의 IP 주소, 원래 요청한 호스트 이름 등)가 Nginx를 거치면서 사라지지 않고 뒷단의 애플리케이션에도 그대로 전달되도록 하는 설정이다. Ubuntu 계열에서는 이 설정 파일을 실제로 활성화하려면 심볼릭 링크를 만들어야 한다.

```
sudo ln -s /etc/nginx/sites-available/lab01.conf /etc/nginx/sites-enabled/
sudo nginx -t          # 설정 문법 검사
sudo systemctl reload nginx
```

`nginx -t`는 설정 파일에 문법 오류가 없는지 미리 검사해주는 명령으로, 이 검사를 통과한 뒤에 `reload`(서비스를 완전히 재시작하지 않고 설정만 다시 읽어 들이는 방식)를 실행하면 서비스 중단 없이 새 설정이 적용된다.

도메인/호스트 매핑

위 설정에서 `server_name`으로 지정한 `lab01.local`은 실제로 등록된 도메인이 아니므로, 이 서버에 접속하는 컴퓨터의 `/etc/hosts` 파일에 이 이름과 서버 IP 주소를 연결하는 항목을 추가해야 브라우저에서 이 이름으로 접속할 수 있다.

```
# /etc/hosts
127.0.0.1 lab01.local lab02.local board.local bank.local
127.0.0.1 attacker.local
```

특히 `attacker.local`이라는 이름은 5장(XSS)과 12장(CSRF)에서 공격자 서버를 가리키는 데 사용되는데, 이렇게 별도의 이름을 붙여 두는 이유가 있다. 만약 공격자 서버도 그냥 `localhost`로 접속한다면, 이 서버가 실습 애플리케이션과 "같은 출처(origin)"인지 "다른 출처"인지가 브라우저의 눈에는 모호해질 수 있다. XSS나 CSRF 같은 공격의 본질은 서로 다른 출처(정상 사이트와 공격자 사이트) 사이에서 발생하는 신뢰의 오남용이므로, 실습에서도 이 둘을 명확히 구분되는 별도의 이름으로 두어야 "출처가 다르다"는 개념을 눈으로 확인할 수 있다.

SELinux 확인

Rocky Linux는 기본적으로 SELinux가 활성화되어 있다. Nginx가 다른 포트(3001)로 프록시하는 것을 허용해야 한다.

```
sudo setsebool -P httpd_can_network_connect on
```

이 작업을 하지 않으면 Nginx가 3001 포트에 연결하지 못해 502 Bad Gateway가 발생할 수 있다.

정적 파일·업로드 디렉터리 권한 설계

7장(파일 업로드)의 실습을 위해, 사용자가 업로드한 파일이 저장될 `/var/lab-uploads` 디렉터리의 소유자와 권한을 미리 설계해 둔다.

```
sudo chown labuser:www-data /var/lab-uploads
sudo chmod 750 /var/lab-uploads
```

이 설정은 애플리케이션을 실행하는 `labuser` 계정에게는 읽고 쓰는 권한을, Nginx가 실행되는 `www-data` 그룹에게는 읽기 권한만 부여하고, 그 외의 계정에게는 아무 권한도 주지 않는다. `750`이라는 숫자는 소유자에게는 읽기·쓰기·실행(7), 그룹에게는 읽기·실행(5), 그 외에는 아무 권한도 없음(0)을 의미하는 리눅스 권한 표기법이다. 이렇게 세밀하게 권한을 설계해 두는 이유는, 7장에서 다룬 "업로드된 파일이 실행 가능한 스크립트로 취급되어 웹shell 공격에 악용되는" 시나리오를 방지하기 위한 근거가 이 권한 설계에 있기 때문이다.

HTTPS 기초와 자체 서명 인증서

12장(CSRF)의 쿠키 속성 실습에서는 `Secure` 속성(HTTPS 연결에서만 쿠키가 전송되도록 하는 속성)을 다루는데, 이를 실제로 시연하려면 서버가 HTTPS를 지원해야 한다. 실습 환경에서는 공인 인증기관의 인증서 대신, 직접 서명한 자체

서명 인증서(self-signed certificate)를 사용해 이 기능을 확인할 수 있다.

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
```

이 명령은 유효기간 365일의 인증서와 그에 대응하는 개인키를 즉석에서 생성해준다. 자체 서명 인증서는 공인 인증기관이 보증하지 않으므로 브라우저가 "안전하지 않은 연결"이라는 경고를 보여주지만, HTTPS 연결 자체는 정상적으로 암호화되므로 **Secure** 쿠키 속성의 동작을 확인하는 교육 목적으로는 충분하다.

0.9 환경 점검

설치 항목 체크리스트

지금까지 설치하고 설정한 항목들이 모두 정상적으로 동작하는지 하나씩 확인하며 이 장을 마무리한다.

```
node -v           # Node.js 설치 확인
npm -v           # npm 설치 확인
systemctl is-active mysql      # MySQL 서비스 동작 확인
systemctl is-active nginx     # Nginx 서비스 동작 확인
sudo ufw status  # 방화벽 규칙 확인 (Rocky: firewall-cmd --
list-all)
curl http://localhost:3001/healthz # 애플리케이션 정상 응답 확인
```

systemctl is-active 서비스이름은 해당 서비스가 현재 정상적으로 실행 중이면 **active**를, 그렇지 않으면 다른 상태 문구를 출력한다. 마지막 줄의 **curl** 명령은 0.7절에서 만든 헬스체크 엔드포인트에 실제로 요청을 보내, 애플리케이션이 요청을 정상적으로 받아 응답할 수 있는 상태인지 최종 확인하는 절차다.

트러블슈팅

포트 충돌: 애플리케이션을 실행했는데 "주소가 이미 사용 중"이라는 오류(**EADDRINUSE**)가 발생하는 경우, 해당 포트를 이미 다른 프로세스가 점유하고 있다는 뜻이다. **ss -tlnp | grep 포트번호** 명령으로 그 포트를 점유한 프로세스를 확인하고, 필요하다면 해당 프로세스를 종료한 뒤 다시 시작한다.

권한 오류(EACCES/EADDRINUSE): 1024번 이하의 포트(예: 80번)를 일반 사용자 권한으로 직접 열려고 하면 권한 오류가 발생한다. 이 교재의 구조에서는 Node.js 애플리케이션이 직접 80번 포트를 사용하지 않고, 항상 3000번대 포트에서 실행된 뒤 Nginx가 80번 포트를 대신 열어 중계하도록 설계했으므로 이 문제는 애초에 피해가도록 되어 있다. 만약 이 원칙과 다르게 구성하다가 이 오류를 만났다면, 애플리케이션이 낮은 번호의 포트를 직접 사용하려 하고 있지 않은지 확인한다.

MySQL 인증 플러그인 오류: 0.5절에서 설명한 것처럼, Node.js 드라이버가 **caching_sha2_password** 인증 방식을 지원하지 않아 연결이 거부되는 경우 오류 메시지에 **ER_NOT_SUPPORTED_AUTH_MODE**와 같은 문구가 포함된다. 이때는 해당 계정의 인증 방식을 **mysql_native_password**로 변경하거나, 드라이버를 이 인증 방식을 지원하는 최신 버전으로 올린다.

SELinux 차단: Rocky Linux 환경에서 Nginx가 뒷단의 Node.js 애플리케이션으로 요청을 전달하지 못하고 502 오류를 반환하는 경우, SELinux가 Nginx 프로세스의 네트워크 연결 시도 자체를 차단하고 있을 가능성이 있다. 다음 명령으로 SELinux가 이 연결을 허용하도록 정책을 조정할 수 있다.

```
sudo setsebool -P httpd_can_network_connect 1
```

이 명령이 실제로 어떤 접근이 차단되었기 때문에 필요한지 확인하려면, `sudo ausearch -m avc -ts recent` 명령으로 최근 SELinux가 거부한 접근 기록을 조회해 원인을 파악한 뒤 대응하는 것이 바람직하다.

Nginx 502 오류: 사용자가 브라우저에서 502 Bad Gateway 오류를 만나는 경우, 이는 Nginx는 정상적으로 요청을 받았지만 그 뒤에 연결하려던 애플리케이션이 응답하지 않았다는 뜻이다. 원인은 대개 두 가지다. 하나는 해당 포트의 Node.js 애플리케이션이 아직 시작되지 않았거나 오류로 죽어 있는 경우이고(`systemctl status lab01`과 `journalctl -u lab01`로 확인), 다른 하나는 방금 설명한 SELinux나 방화벽이 Nginx에서 해당 포트로의 연결 자체를 막고 있는 경우다.

이 장에서 만든 서버는 이후 13개 장에 걸쳐 계속 사용되는 공통 토대다. 여기서 설정한 디렉터리 구조, 계정 권한, 방화벽 규칙, 스캐폴드 코드가 흔들림 없이 자리 잡혀 있어야, 다음 장부터는 매번 환경을 설명하는 데 시간을 쓰지 않고 곧바로 그 장의 취약점 이야기로 들어갈 수 있다.